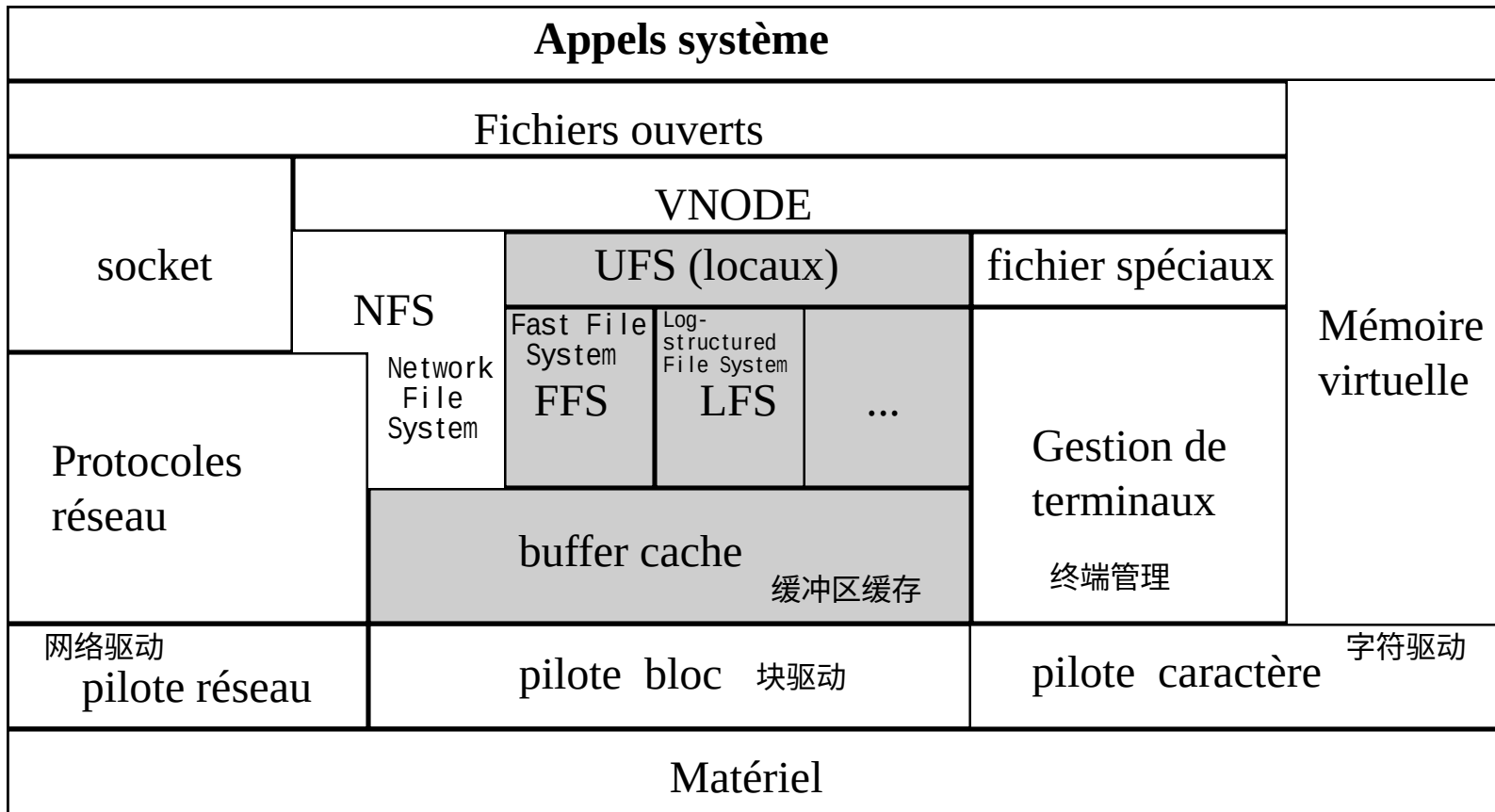


Systeme de gestion des Entrées/Sorties

1- Le sous système d'entrées/sorties

2- Les systèmes de fichiers locaux

I - Système de fichiers locaux

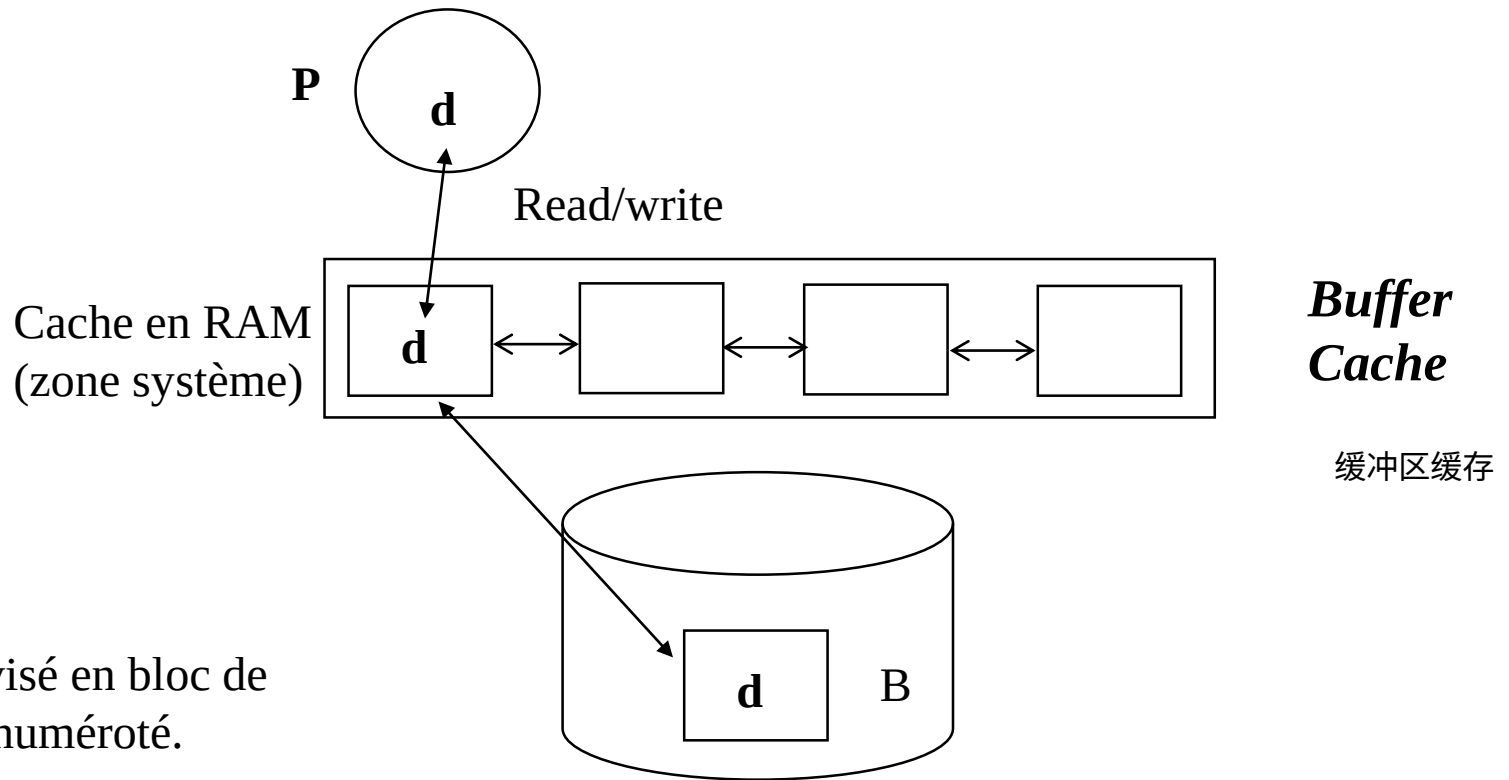


PARTIE 1 : Cache

Principe du cache

外部
|
缓存
|
硬盘块

Accès donnée **d** dans bloc B



Disque divisé en bloc de
taille fixe numéroté.

1 - Buffer Cache

- Principe:
 - Les lectures/écritures par blocs
 - Les blocs sont conservés en mémoire dans une zone du système = buffer cache
- Avantages:
 - Limiter le nombres d'E/S (localité)
 - Dissocier E/S logique et E/S physique (asynchronisme)
 - Anticipation en cas d'accès séquentiel
- Inconvénient:
 - Risque d'incohérence (perte de données) en cas de défaillance

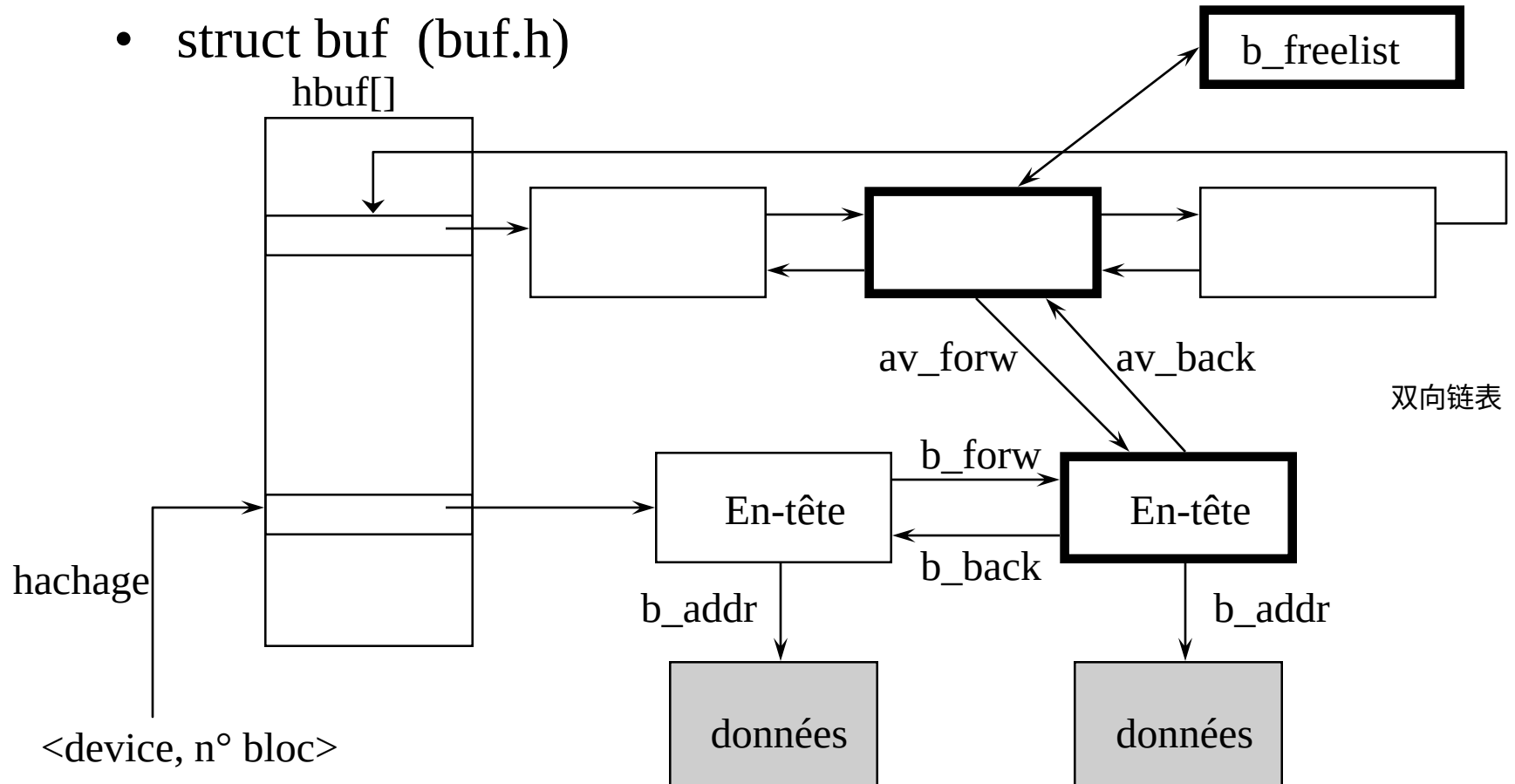
逻辑IO：软件层面的输入输出

物理IO：硬件层面的输入输出

Structure générale

空闲列表

- struct buf (buf.h)



En-tête du buffer cache

Quelque champs *struct buf*

b_flags : états du bloc

- | | |
|---|-------------|
| – *b_forw : pointeur buffer suivant dans le même pilote | chaînage b_ |
| – *b_back : pointeur buffer précédent dans le même pilote | |

- | | |
|---|--------------|
| – *av_forw : pointeur buffer libre suivant (dans la b_freelist) | chaînage av_ |
| – *av_back : pointeur buffer libre précédent | |

- | | |
|------------------------------------|----------------------|
| – b_dev : numéro device | identification block |
| – b_blkno : numéro logique du bloc | |

- | | |
|--------------------------------------|--------------|
| – b_addr : pointeur vers les données | block donnée |
|--------------------------------------|--------------|

...

Etats d'un buffer

- Valeurs du champs b_flags
- Disponible : pas d'E/S en cours => **dans la b_freelist**
- Indisponible (B_BUSY positionné dans b_flags)
 - B_DONE : E/S terminée (**contenu est correct**)
 - B_ERROR : E/S incorrecte
 - B_WANTED : désiré par un processus (réveiller en fin E/S)
 - B_ASYNC : ne pas attendre fin E/S (E/S asynchrone)
 - B_DELWRI : retarder l'écriture sur disque (tampon «sale»)

哈希表，
使用哈希函数来确定设备和块的组合应该存储在哪个位置或slot。

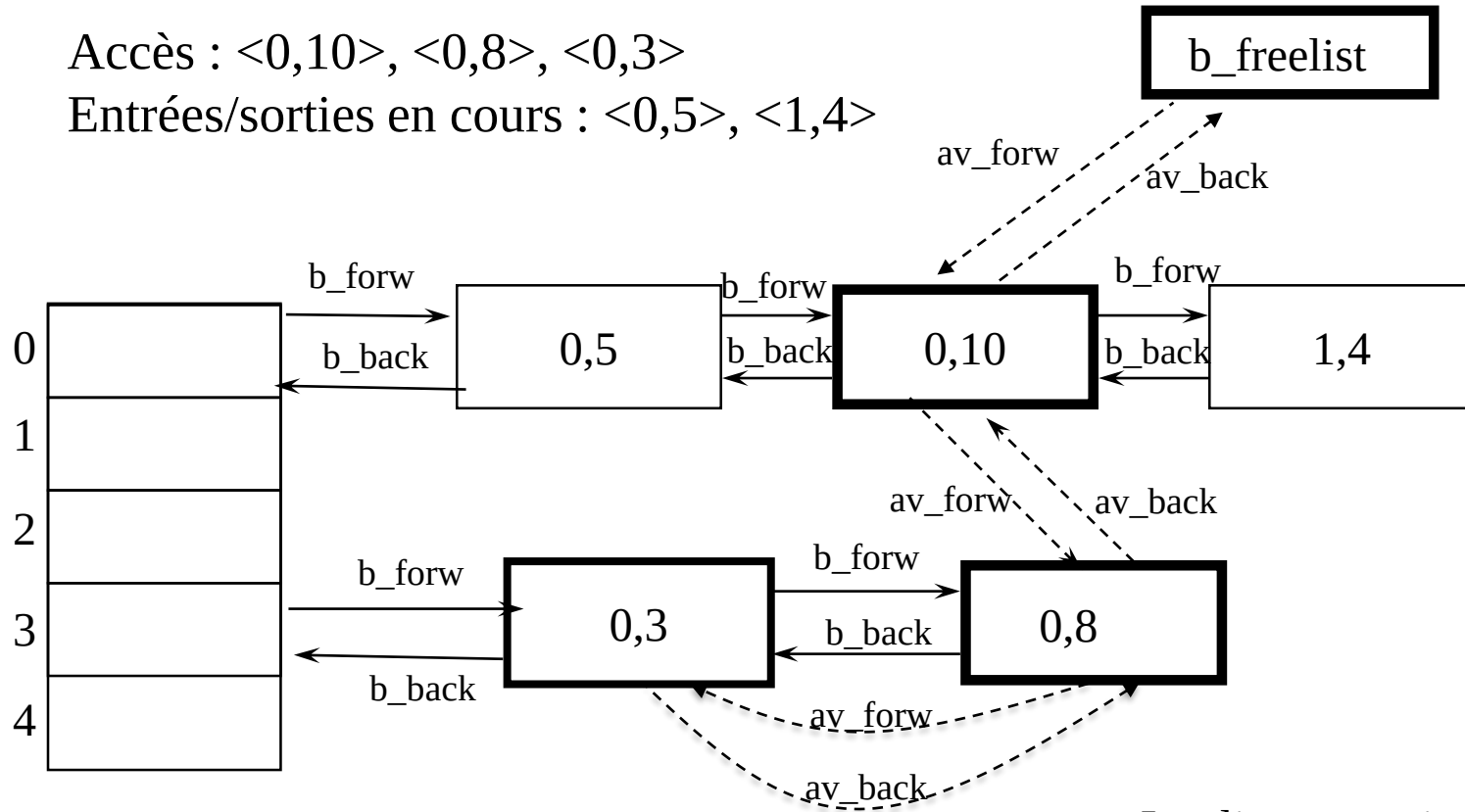
slot：哈希表中的位置/索引

Exemple

设备0, 块10 $bhash(dev, bloc) = (dev + bloc) \% 5$ 决定储存位置

Accès : <0,10>, <0,8>, <0,3>

Entrées/sorties en cours : <0,5>, <1,4>



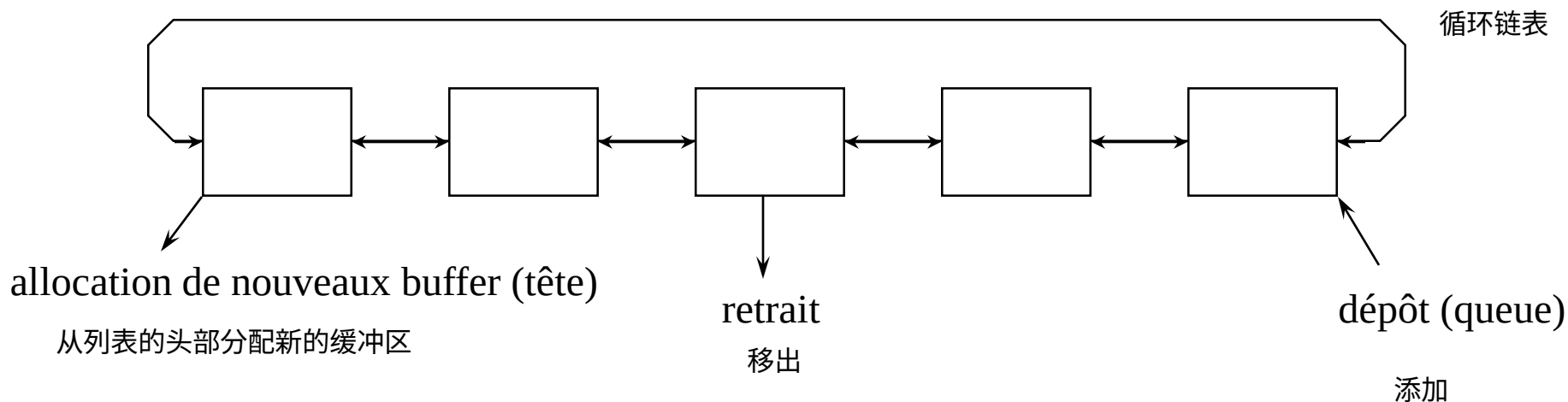
Les liste sont circulaire

Les primitives

- Recherche ou allocation d'un buffer : **getblk**
- Libération d'un buffer : **brelse**
- Lecture d'un bloc : **bread**
- Lecture par anticipation d'un bloc : **breada**
- Ecriture différée d'un bloc : **bdwrite** (buffer *delayed* write)
- Ecriture asynchrone^{异步}: **bawrite**
- Ecriture synchrone_{同步} : **bwrite**

Gestion des tampons

- Liste des buffer libres : listes circulaires avec gestion LRU



- Accès à un buffer par hash-coding
 - `bhash(b_dev, b_blkno)` 哈希函数
 - Distribution uniforme des tampon dans les files

Recherche/Allocation de buffer (getblk)

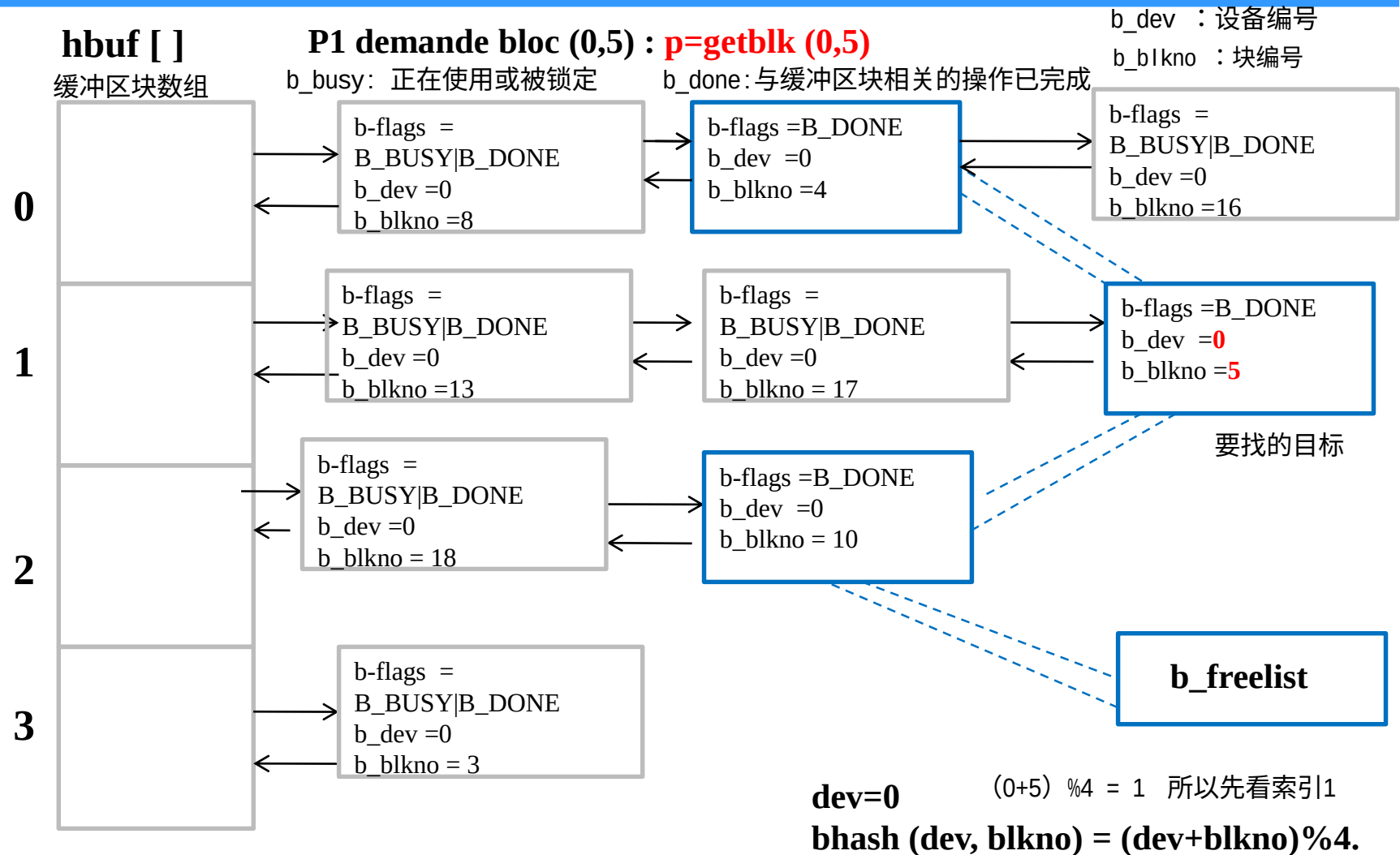
Entrées : numéro de bloc, device

```
→ Tant que (tampon non trouvé)      循环, 直到找到缓冲区
    Si (bloc dans file indique par bhash(bloc, device) ) {  如果通过哈希函数bhash指定的队列中存在该块
        Si (état buffer = B_BUSY) {
            marquer le buffer B_WANTED
            sleep(tampon libre);    休眠直到有一个缓存区被释放
            continue;
        }
        marquer le buffer B_BUSY, le retirer de la b_freelist  将改缓存区表示为bbusy, 并从b_freelist中移除
        Retourner le buffer;
    Sinon { // Le bloc n'est pas dans le buffer cache          如果不在哈希函数指定的队列中
        Si (b_freelist vide) { // Plus de tampon libre !      如果b_freelist为空
            marquer la b_freelist B_WANTED;
            sleep(un tampon se libère);    休眠直到有一个缓存区被释放
            continue;
        }
        Etat buffer tête = B_BUSY; Retirer le buffer de la b_freelist;
        Si (B_DELWRI positionné) { // le tampon est «sale»   如果缓存中包括未写入磁盘的数据
            écriture asynchrone sur disque;    将其写入硬盘中
            continuer;
        }
        Placer le buffer dans la file correspond au couple <bloc, device>  将缓存放入<bloc,device>对应的队列中
        Retourner le buffer;
    }
}
```

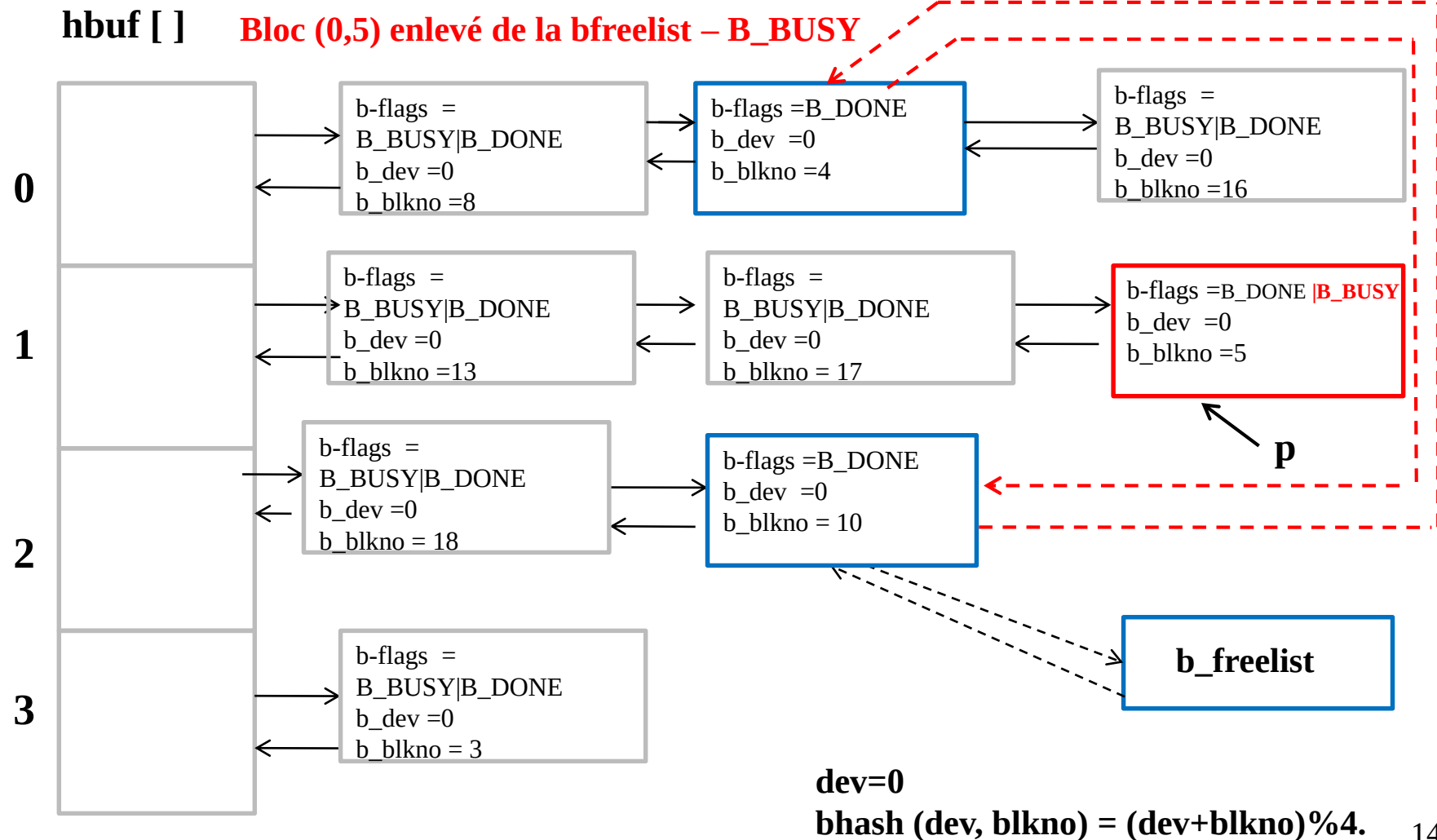
这一页描述了如何使用getblk函数来请求一个特定的块。
例如，当进程P1请求块(0,5)时，它会调用p = getblk(0,5)。此函数首先使用bhash函数来查找是否存在请求的块。如果存在，它将返回该块；如果不存在，它可能会从b_freelist中分配一个新的块，并根据需要更新该块的状态和信息。

1. 先在缓冲区中寻找 (hbuf[])所指示的)
2. 找不到则从b_freelist中取一个

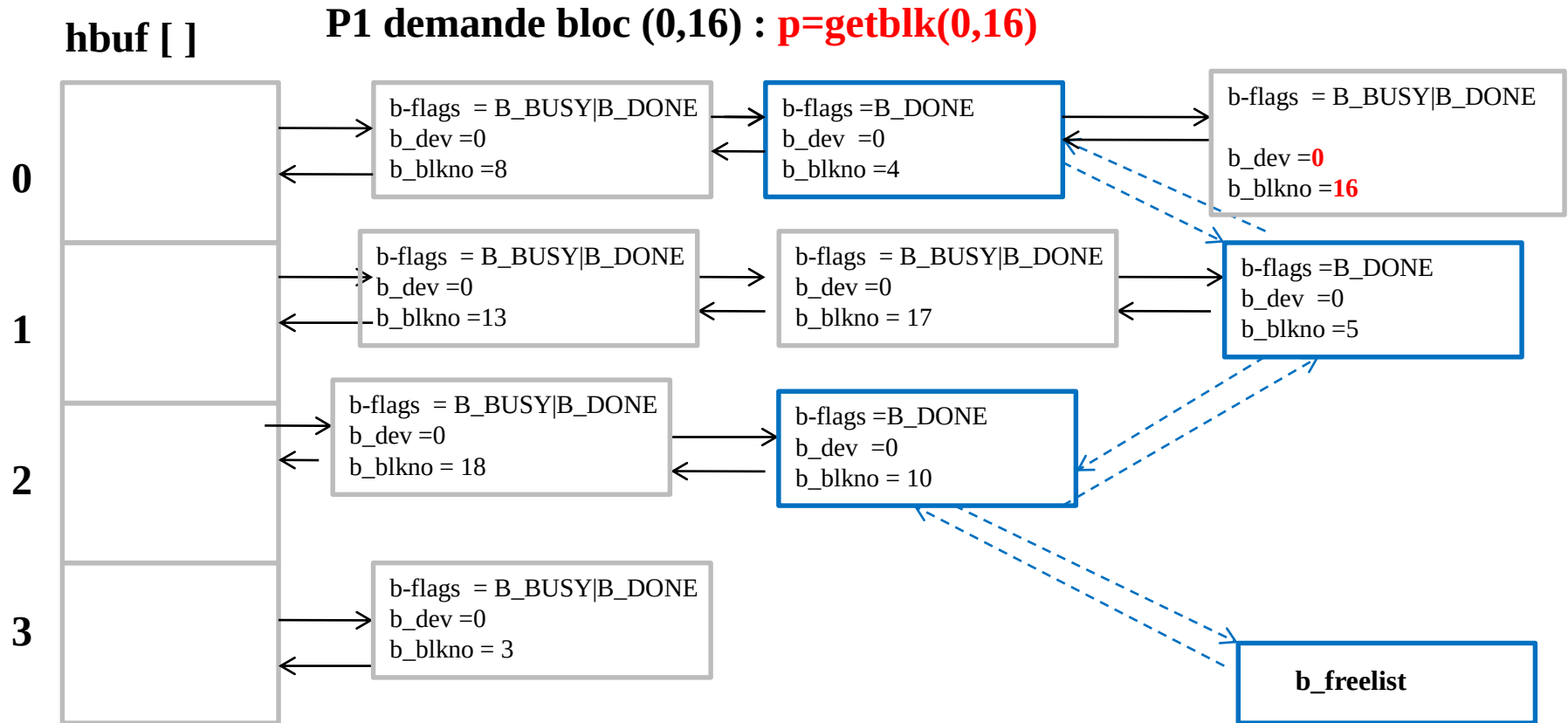
getblk : exemple 1



getblk : exemple 1



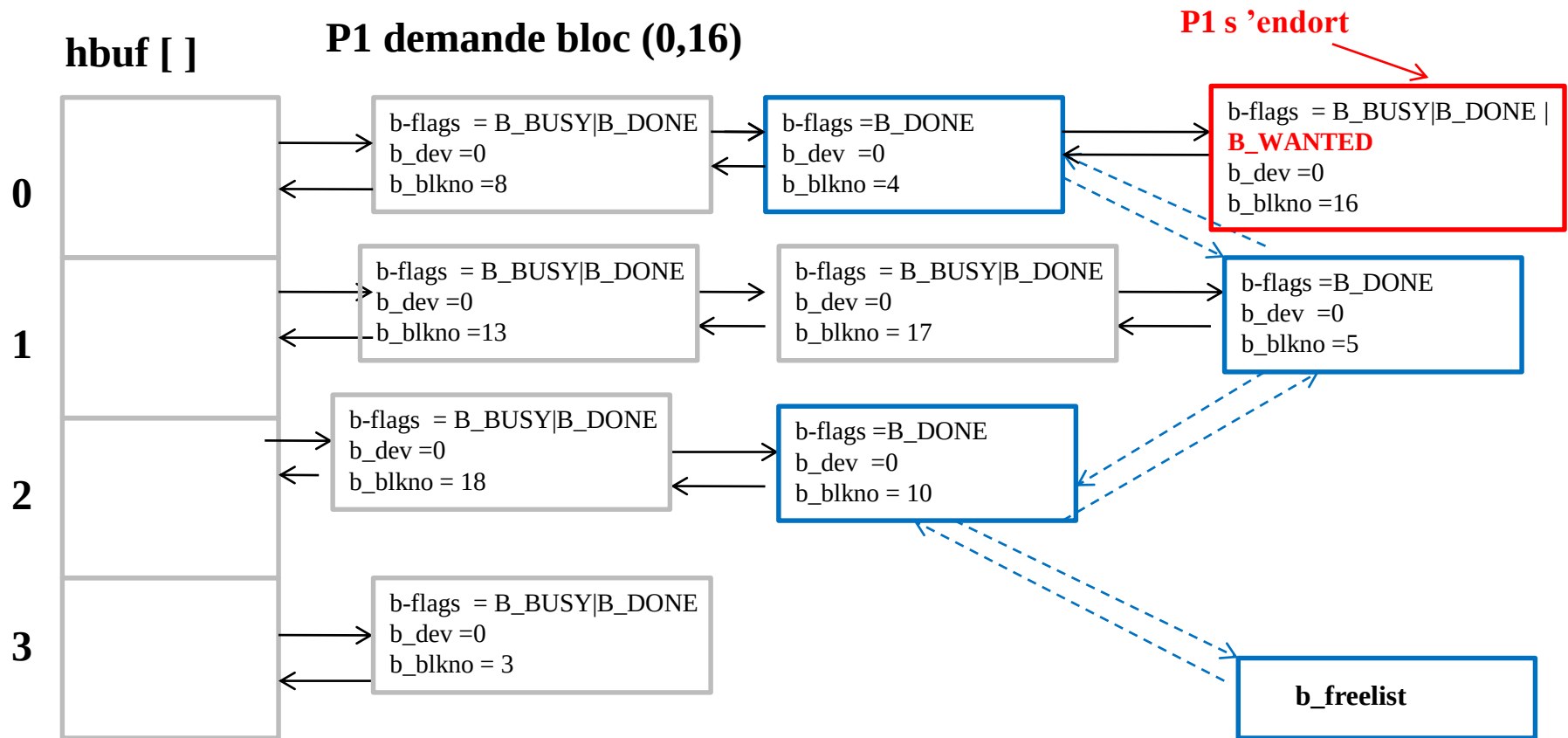
getblk : exemple 2



dev=0

$$\text{bhash}(\text{dev}, \text{blkno}) = (\text{dev} + \text{blkno}) \% 4.$$

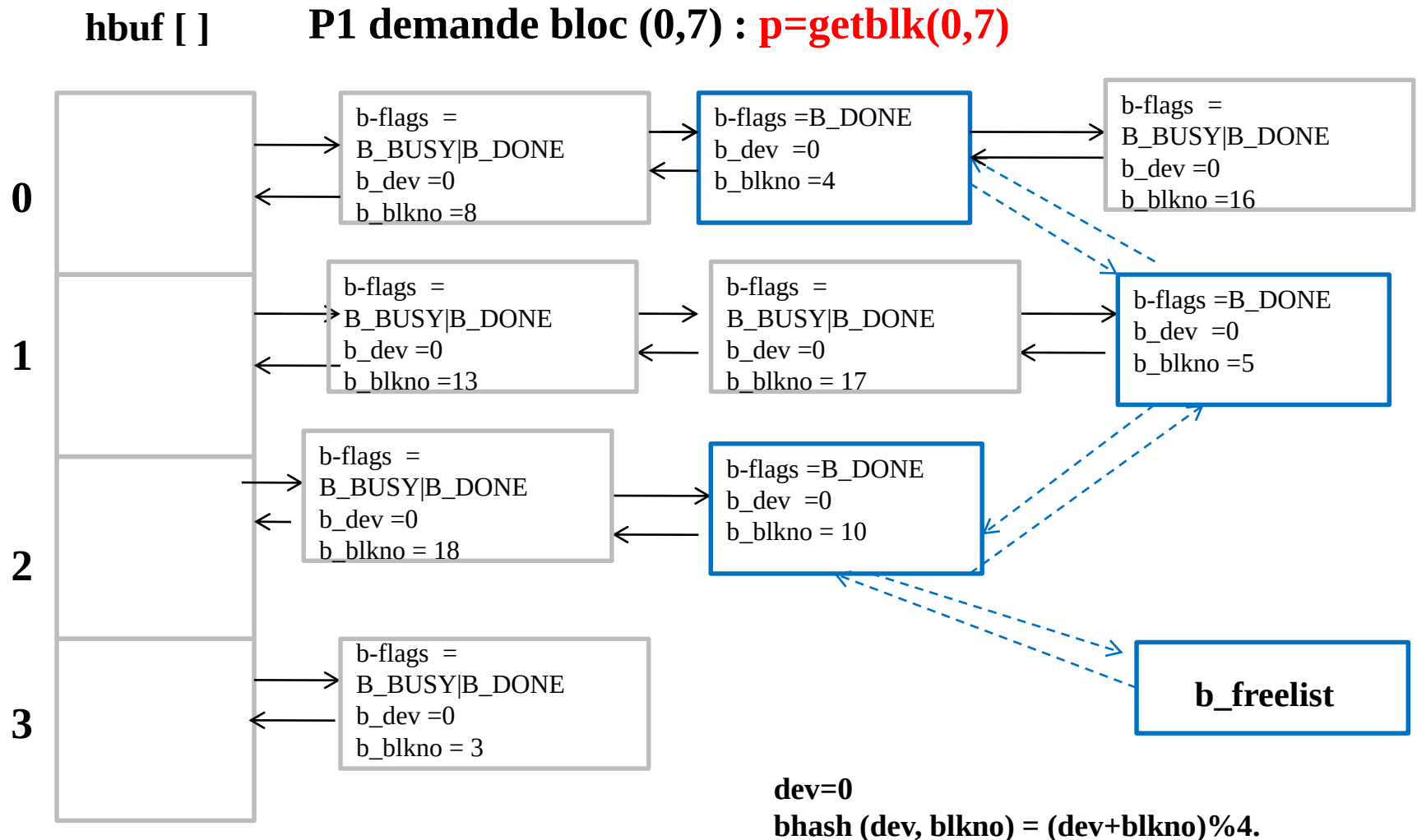
getblk : exemple 2



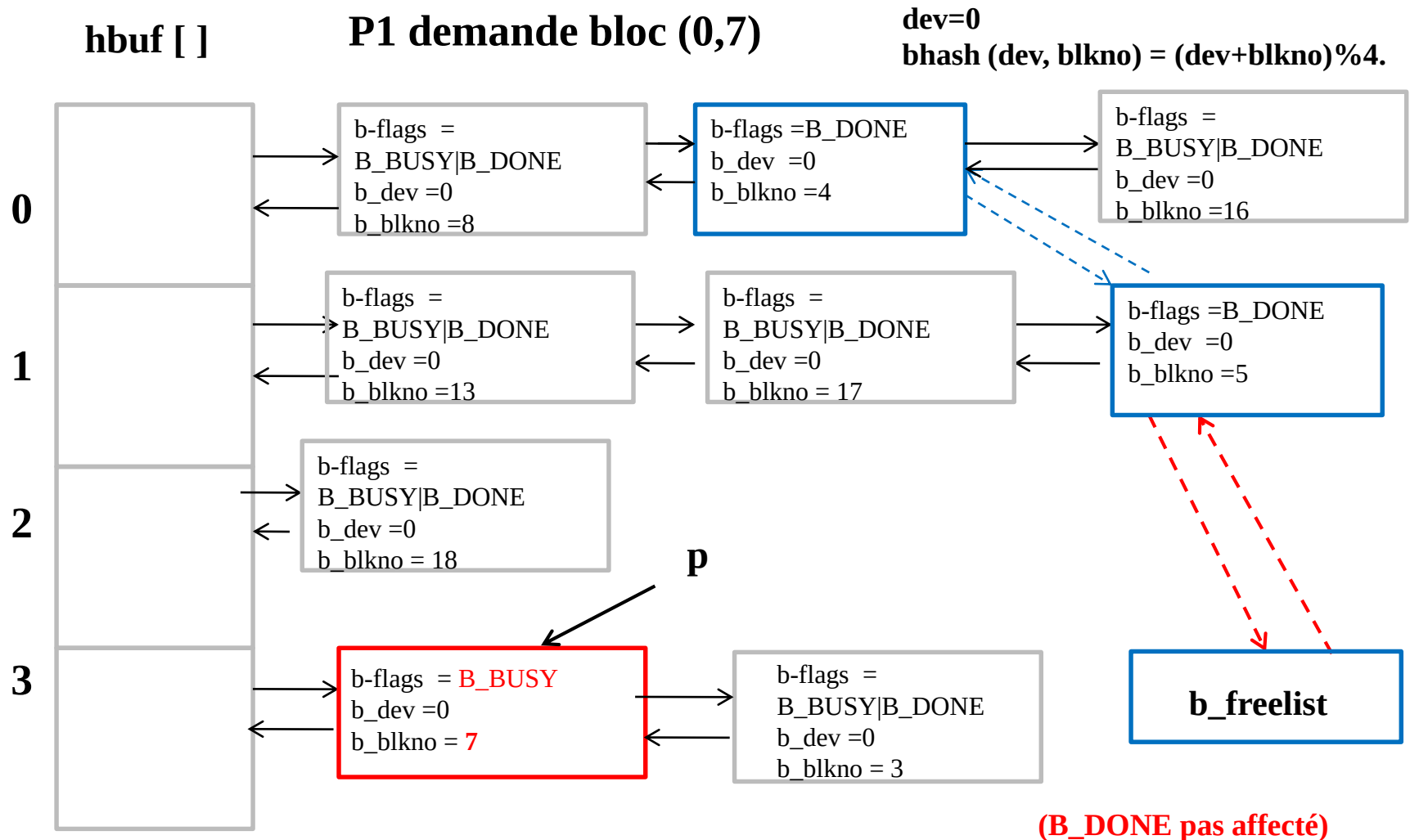
dev=0

bhash (dev, blkno) = (dev+blkno)%4.

getblk : exemple 3



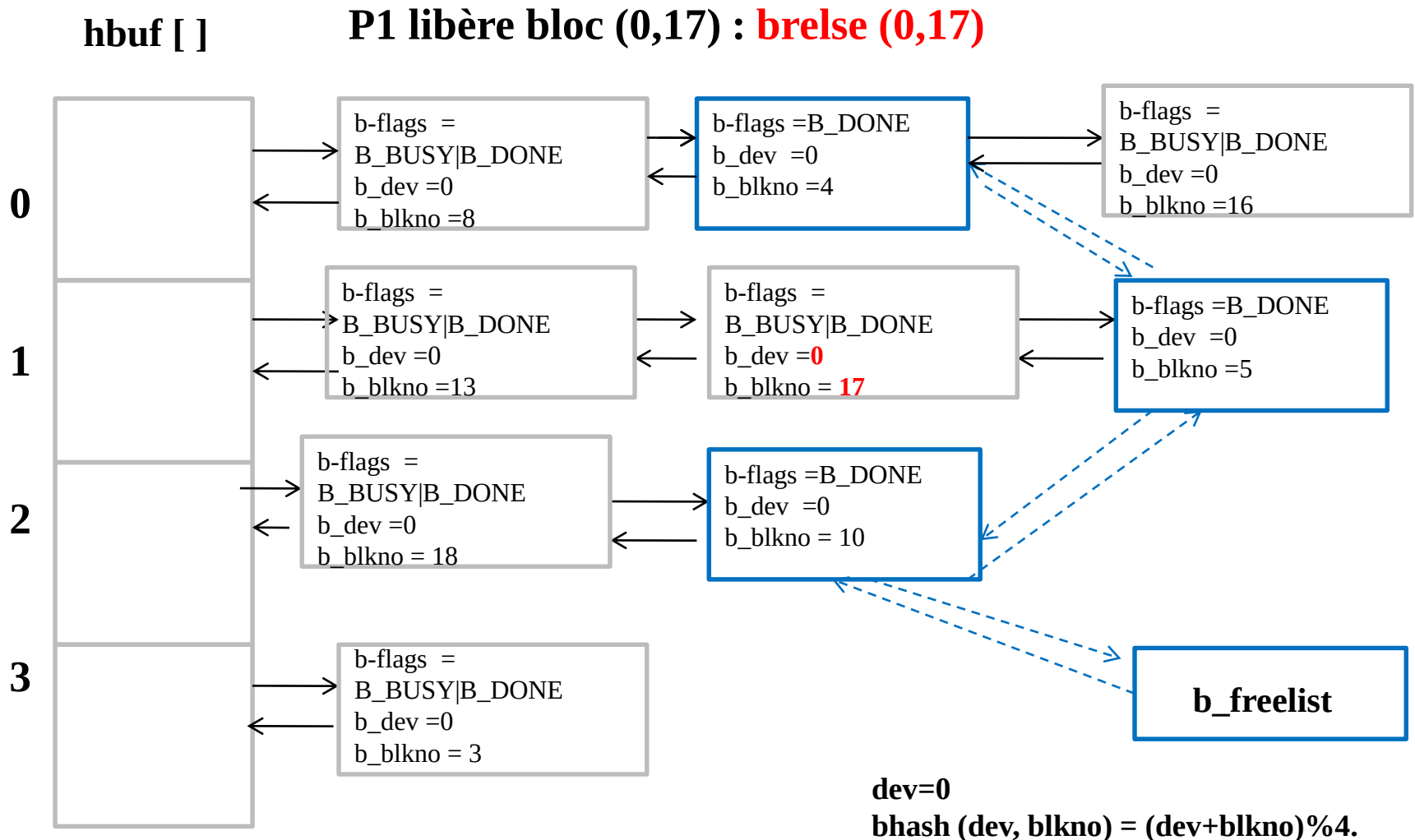
getblk : exemple 3



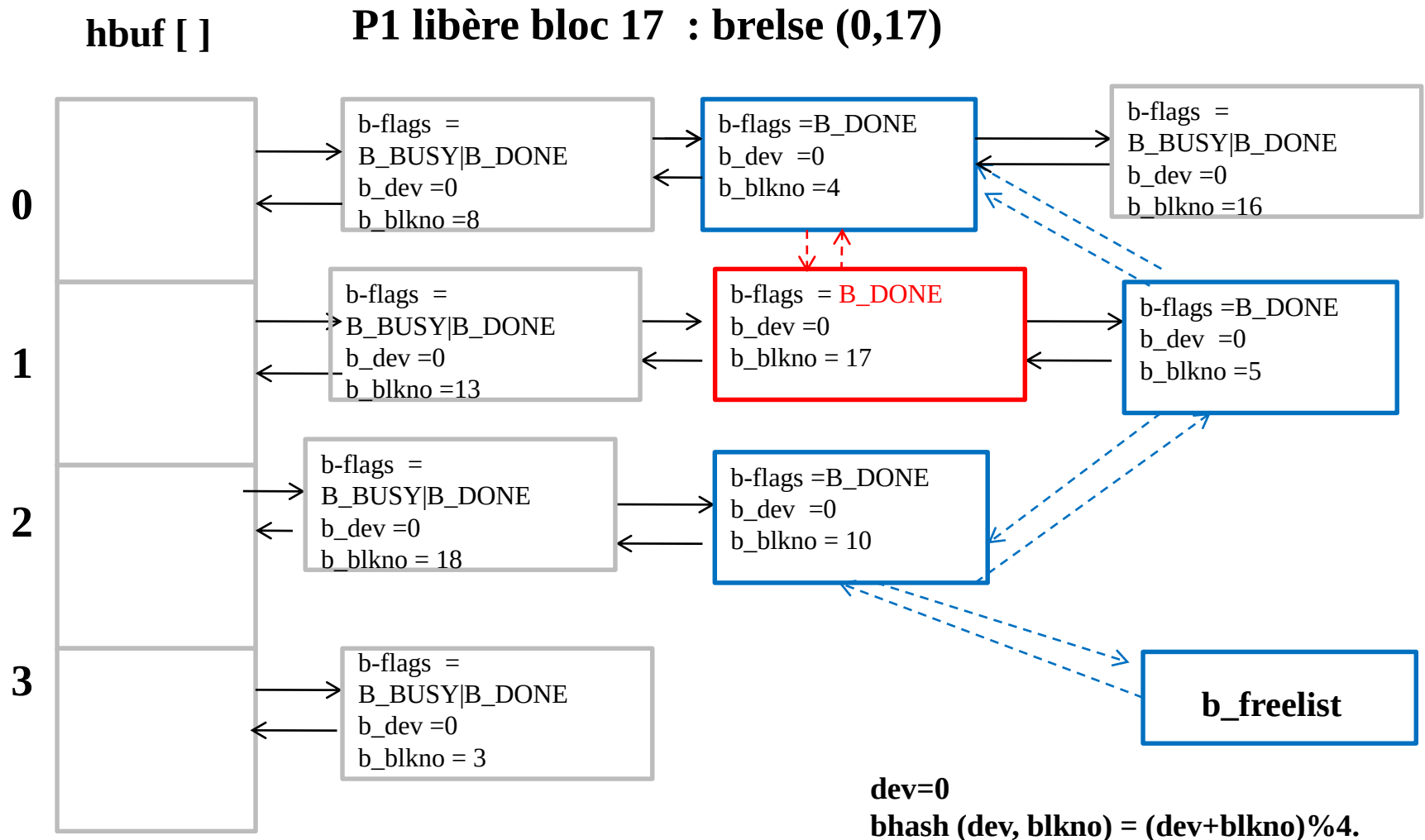
Libération d'un buffer (brelse)

- Réveiller tous les processus en attente qu'un buffer devienne libre (tête b_freelist)
- Réveiller tous les processus en attente que ce buffer devienne libre
- Masquer les interruptions
- Si (contenu du buffer valide)
 - mettre le tampon en queue de la b_freelist
- Démasquer les interruptions
- retirer bit B_BUSY

brelse : exemple



Libération d'un buffer (brelse)



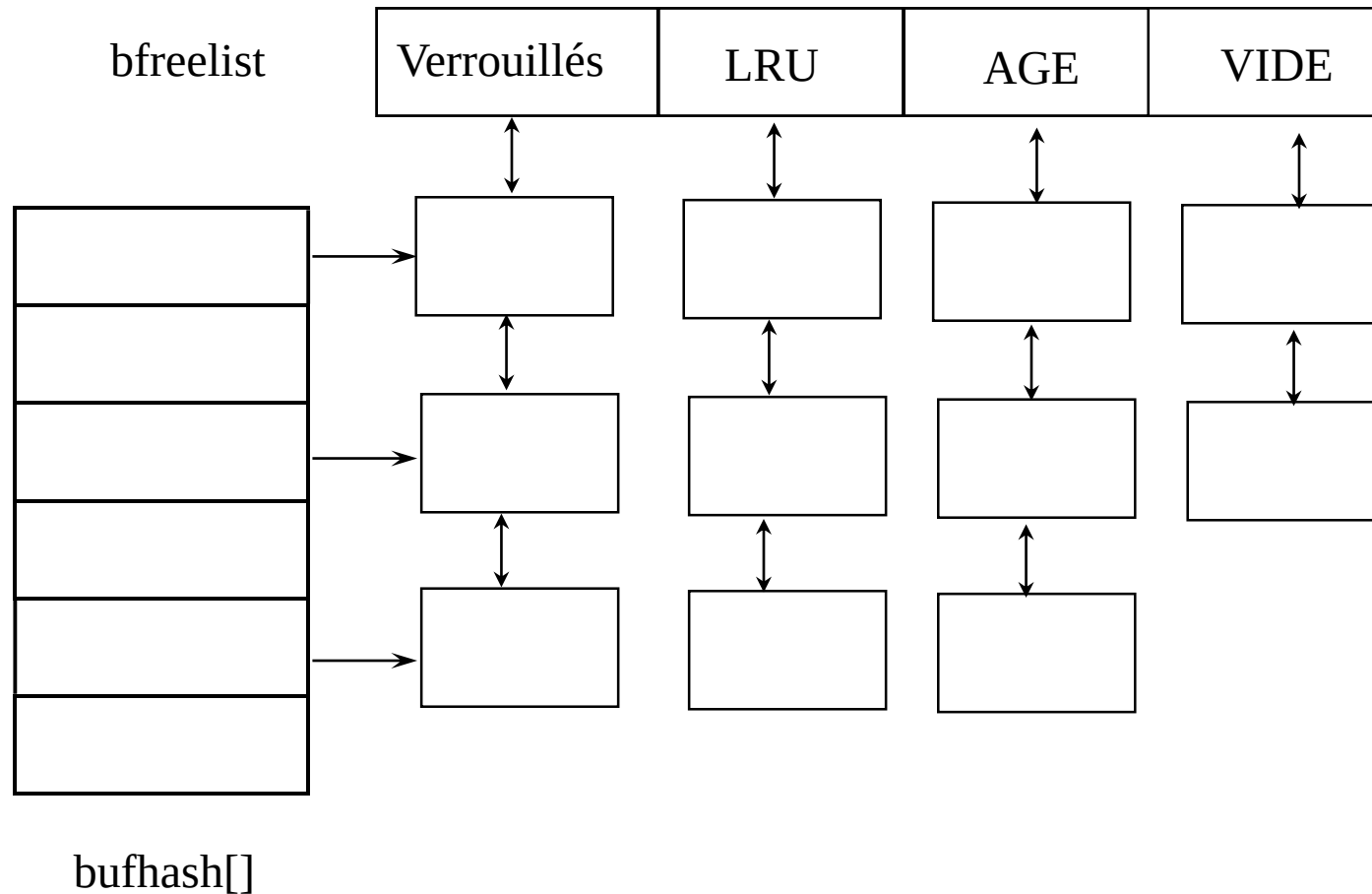
Lecture d'un buffer (bread)

- **Entrées** : device, bloc
- Rechercher ou allouer le bloc (getblk)
- Si (buffer valide et B_DONE)
 retourner le tampon
- Lancer une lecture sur disque (appel du pilote - strategy)
- sleep(attente fin E/S)
- retourner le buffer

Écriture d'un buffer sur disque

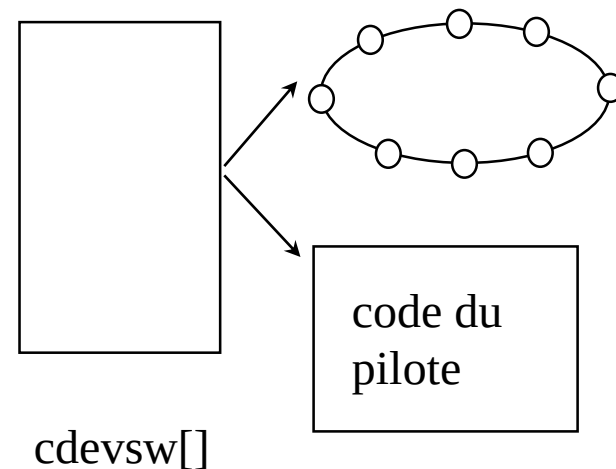
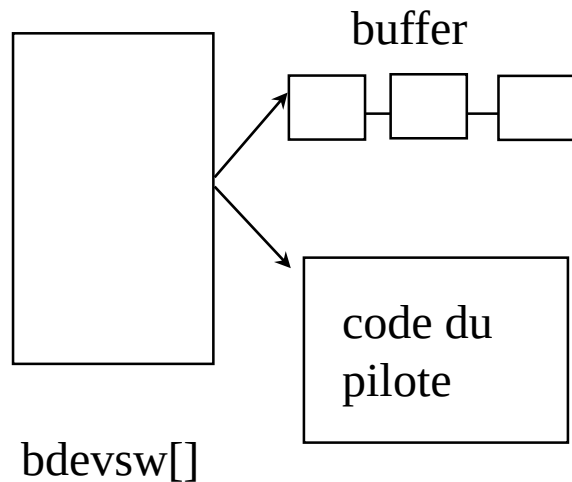
- Bdwrite
Positionnement de B_DELWRI pour E/S asynchrone
- Besoin de place
Dans getblk: Si le bloc n'est pas dans le cache
=> allouer un nouveau buffer
Si B_DELWRI => Écriture
- Régulièrement **sync** parcourt la liste des buffers
Si B_DELWRI => Écriture

Organisation du buffer cache (BSD)



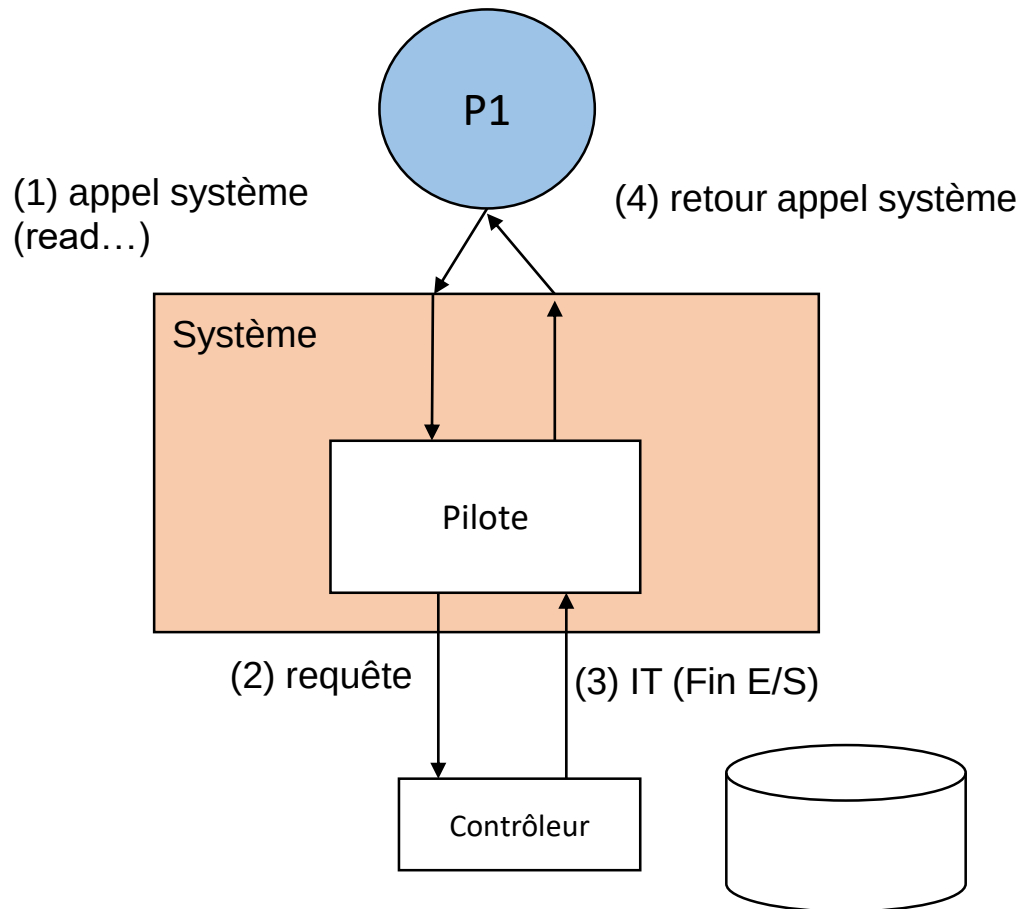
2 - Les Entrées/sorties

- Les types de périphérique
- Mode bloc : accès direct + structuration en bloc
- Mode caractère : accès séquentiel, pas de structuration des données (flux)



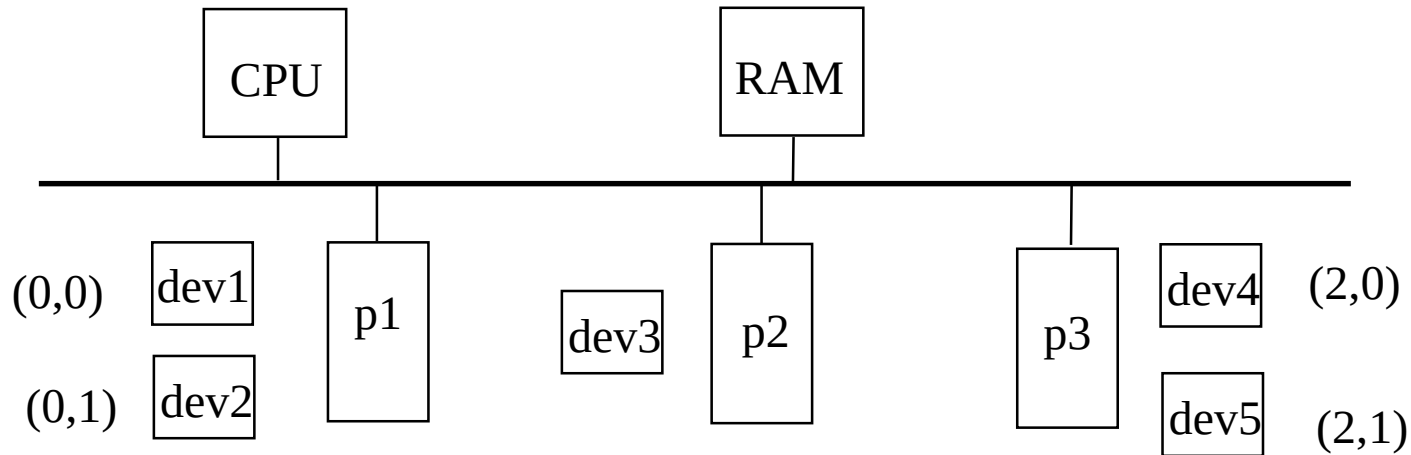
Pilote

- un pilote (driver) = un programme dans le système qui gère le dialogue avec les périphériques



Les pilotes de périphérique

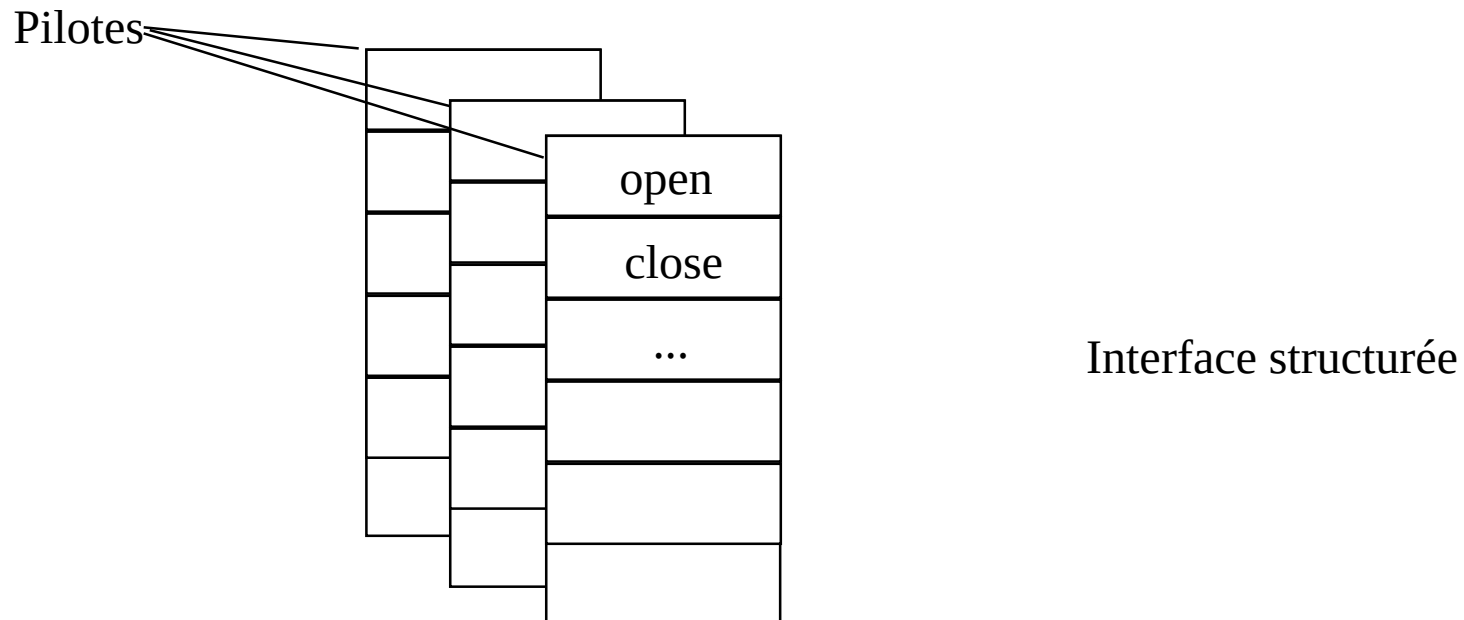
- Configuration typique



- Adresse logique :
 - majeur : numéro de pilote
 - mineur : numéro d'ordre de l'unité logique

Les tables internes

- Pour chaque pilote un ensemble de fonction (points d'entrées)
- Une table pour chaque pilote (device switch)



- 2 types de tables : bdevsw (bloc), cdevsw (car.)

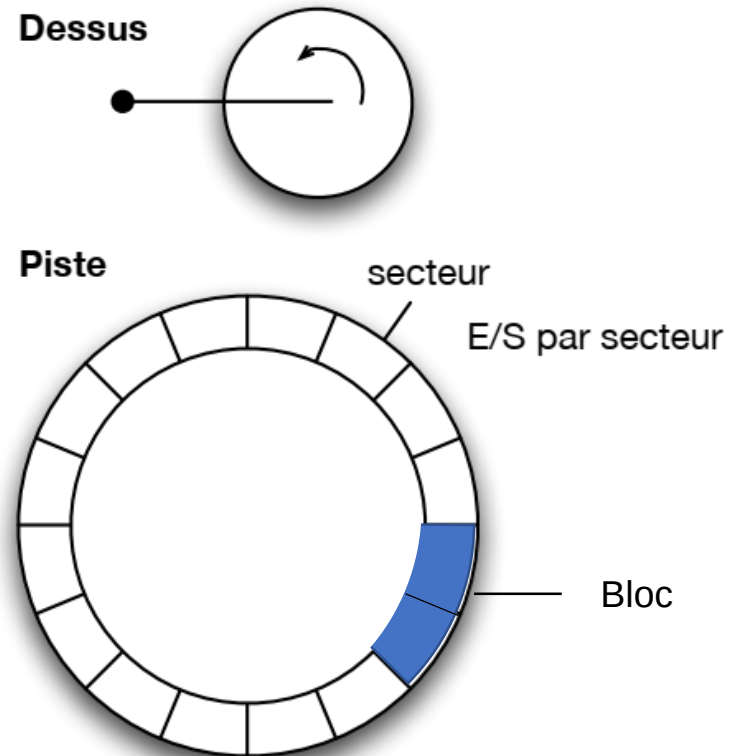
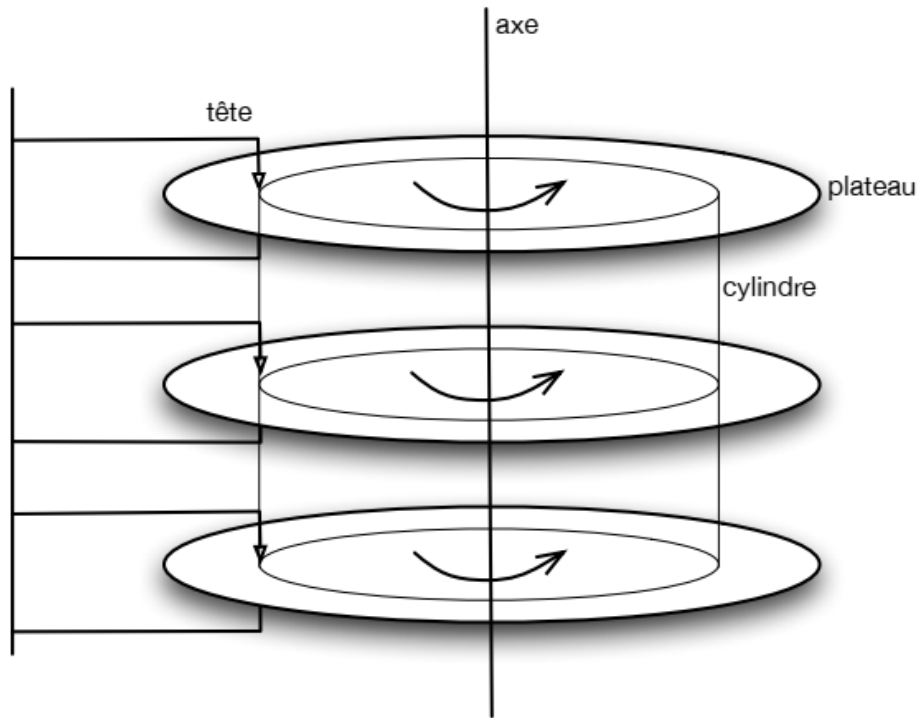
Pilotes en mode bloc

- Table bdevsw :

```
struct bdevsw {  
    int (*d_open)();           /* ouverture */  
    int (*d_close)();          /* fermeture */  
    int (*d_strategy)();        /* Tranfert : Lecture/Ecriture */  
    int (*d_size)();            /* Taille de la partition */  
    int (*d_dump)();            /* Ecrire toute la mémoire physique  
                                sur périphérique */  
    ( int *d_tab;               /* Pointeur vers tampon */ )  
    ...  
} bdevsw[];
```

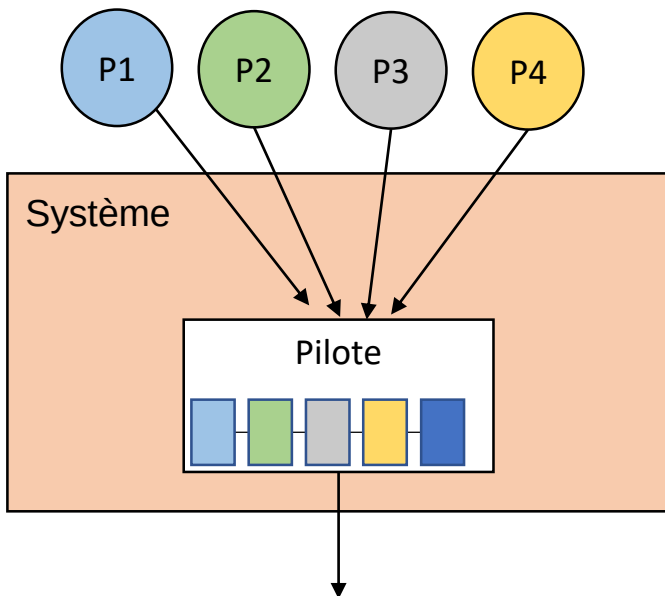
```
(*bdevsw[major(dev)].d_open)(dev, ...);
```

Architecture d'un disque



Accès aux données d'un disque

- Accès aux données : piste (cylindre), secteur
 - Positionnement sur la piste (*seek time*) ;
 - Attendre le passage du secteur sous la piste (latency)
 - Transfert.
- Le pilote du disque gère **une file des requêtes**



Requêtes d'E/S

- Algorithme ascenseur (C_LOOK)
- => limiter les déplacements de têtes

position courante



30	34	35	50	100	150
----	----	----	----	-----	-----

liste des requêtes **après** la position courante

2	7	15	17	20	26
---	---	----	----	----	----

liste des requêtes **avant** la position courante

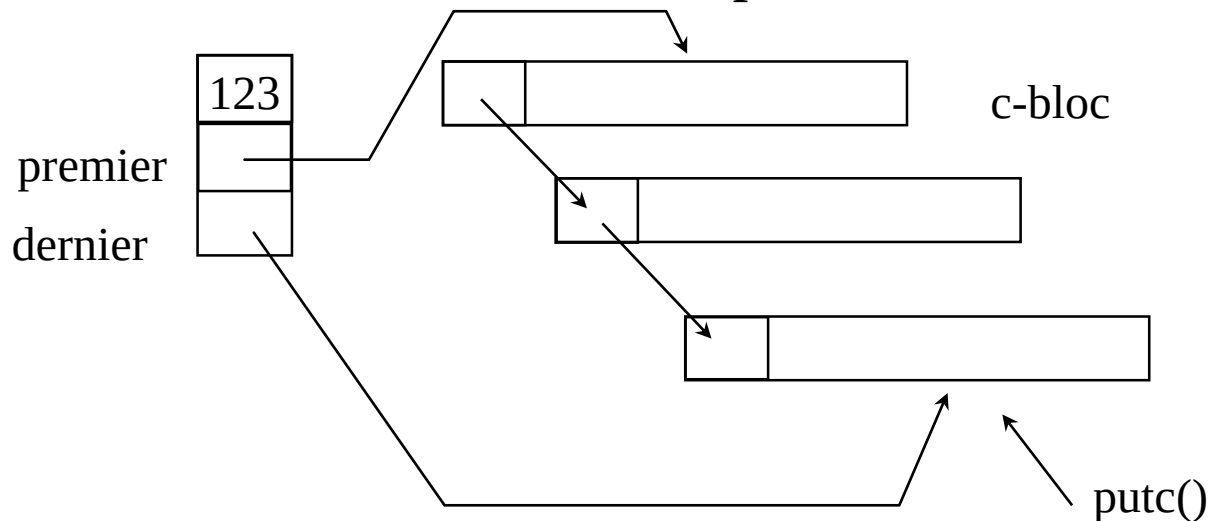
Pilotes en mode caractère

- table cdevsw

```
struct cdevsw {  
    int (*d_open)();  
    int (*d_close)();  
    int (*d_read)();           /* lecture */  
    int (*d_write)();          /* ecriture */  
    int (*d_ioctl)();          /* contrôle */  
    ...  
} cdevsw[];
```

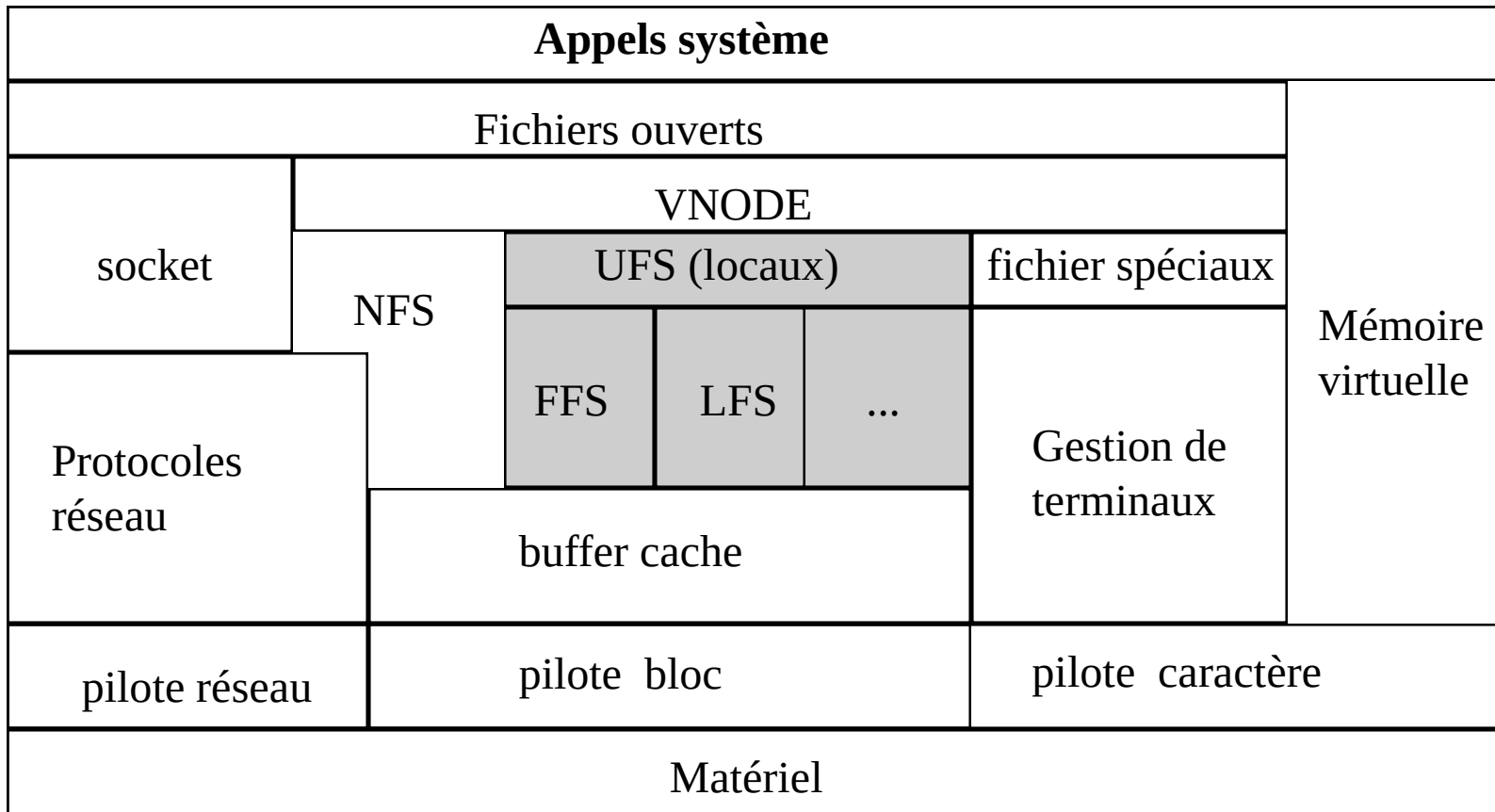
Tampon

- Terminaux : une c-list par terminal



- Les caractères sont copiés vers le contrôleur soit par le processeur soit par le contrôleur (DMA)
- Chaque type de périphérique gère ses propres tampons (possibilité de transférer directement depuis espace util.)

PARTIE 2 : Systèmes de Fichiers



Les différents systèmes de fichiers

- 2 principaux systèmes de fichiers locaux :

System V File System (s5fs)

- Système de fichier de base (78)

Fast File System (FFS)

- Introduit dans 4.2BSD

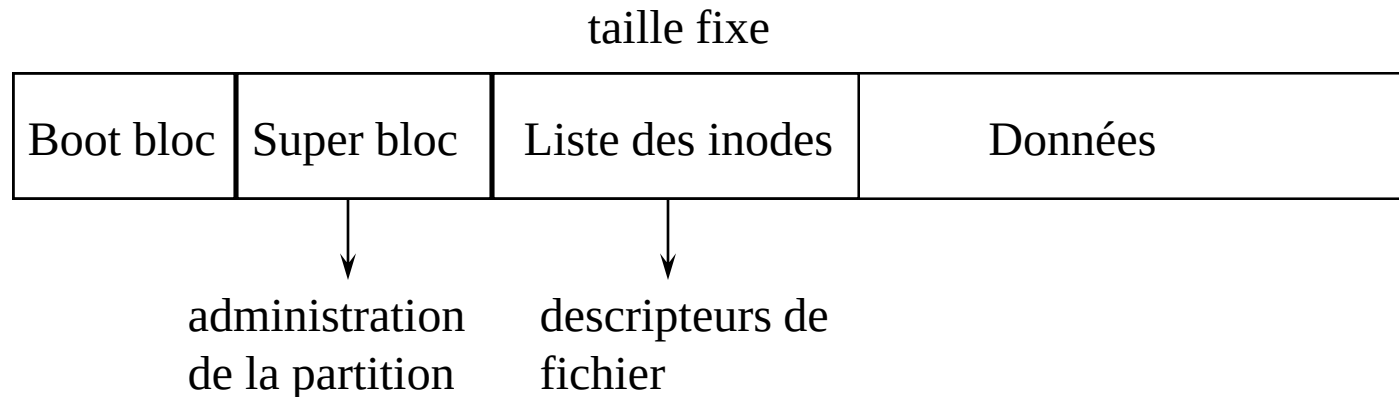
- De nombreux autres systèmes de fichiers :

- Ext2fs (linux, FreeBSD) ...

Organisation générale du disque (s5fs)

- Disque divisé en partitions
 - Chaque partition possède ses propres structures

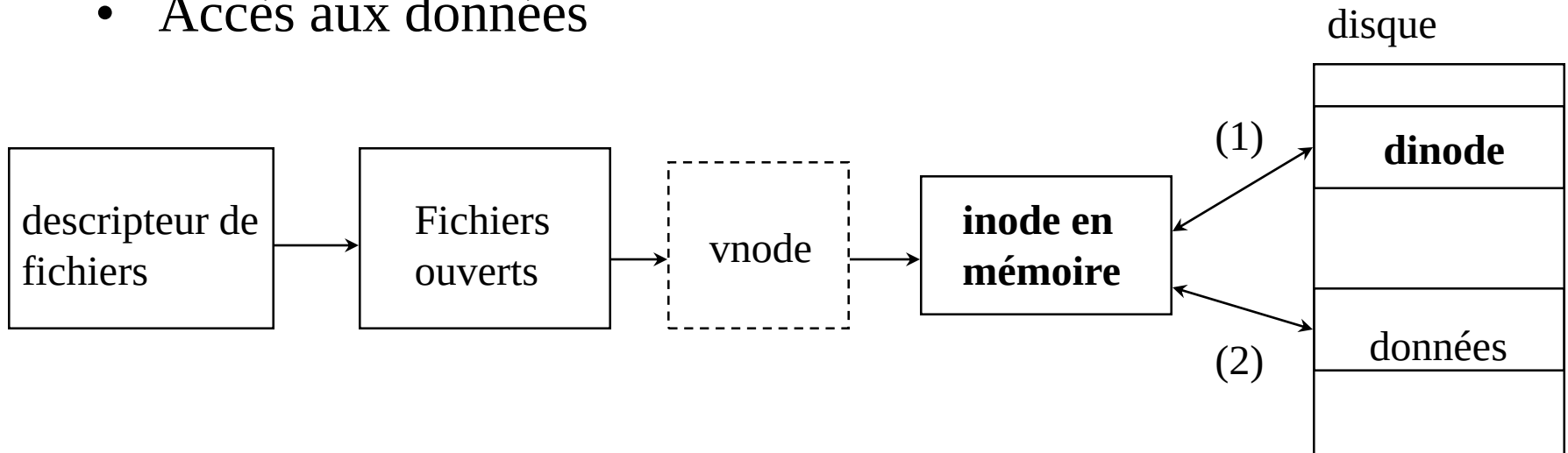
- Organisation d'une partition :



- Numéro d'inode => accès aux blocs du fichier
- Le superbloc contient la liste des blocs libres, liste des inodes libres

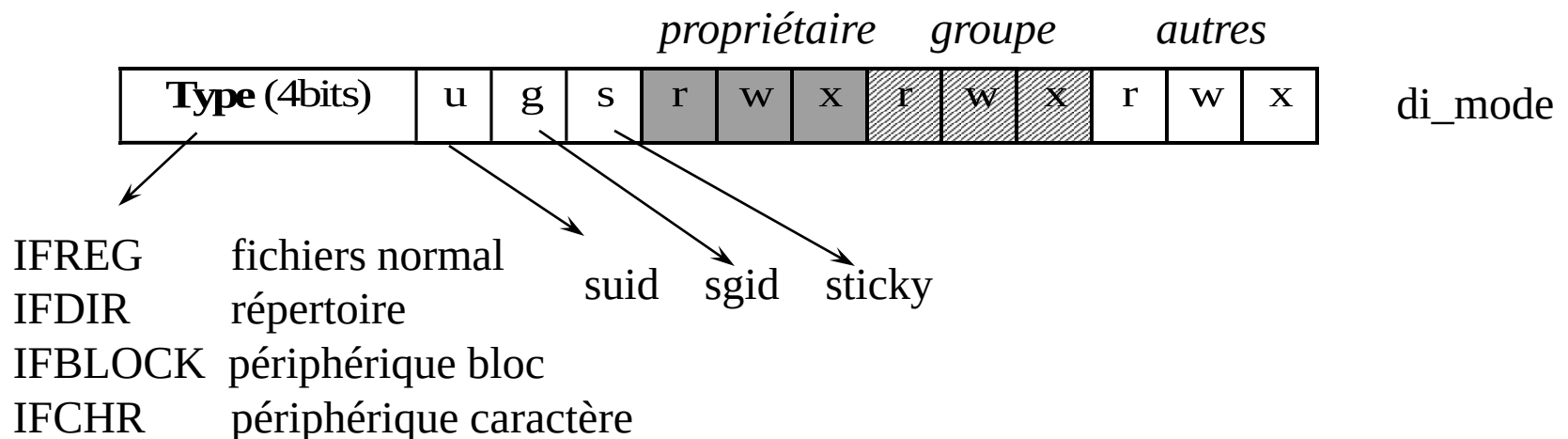
Les structures

- Accès aux données



Accès au données - inode

- Structure des **inodes** = caractéristiques du fichier
- **Sur disque** : struct dinode
 - di_mode : type + droits
 - di_nlink nombre de liens physique
 - di_uid, di_gid
 - di_addr : table de blocs de données
 - di_atime, di_mtime, di_ctime : dates consultation, modification, modification inodes

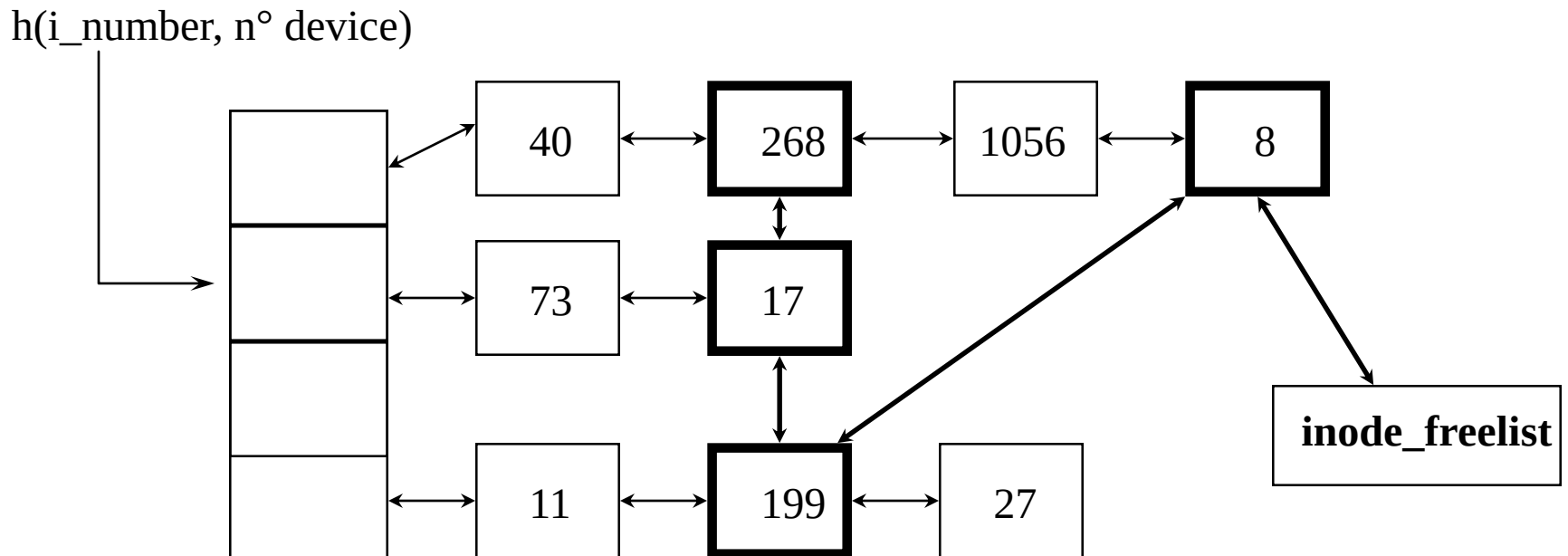


Structure inode

- **En mémoire** : struct **inode**
 - dinode avec en plus :
 - i_dev : device (partition) }
 - i_number : numéro d'inode } accès à l'inode sur disque (mises à jour)
 - i_flags : Flags (synchronisation cache)
 - pointeurs sur la freelist (liste des inodes libres)

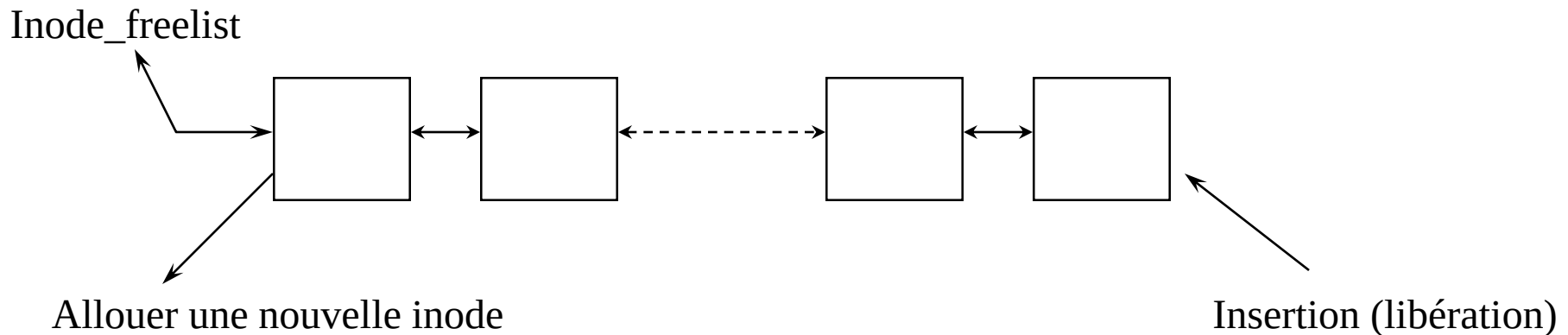
Les inodes en mémoire

- Les entrées de la table des inodes sont chaînées
- Pour trouver rapidement une inode à partir de son numéro utilisation d'une fonction de hachage



Gestion des inodes libres en mémoire

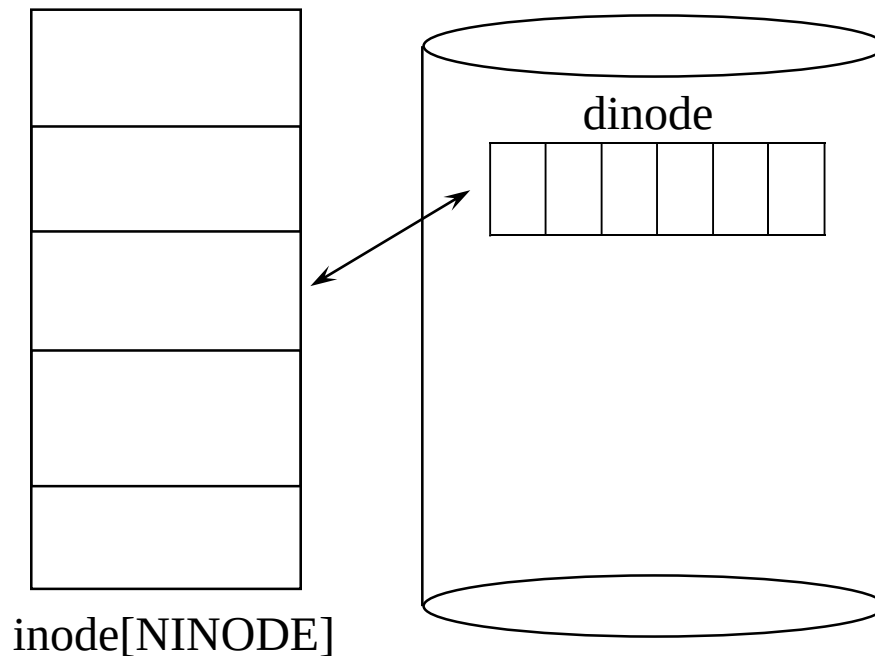
- Si une inode n'est plus utilisée par aucun processus => insertion dans inode_freelist.
- Inode_freelist = cache des anciennes inodes



- Gestion LRU (Least Recently Used) SVR3 (autre critère dans SVR4)

Les inodes en mémoire (Système 5 (V6 et V7) – TD)

Table : inodes pas chaînées : pas de freelist



Structure inode/dinode (TD)

FICHER INODE.H

```
1 #define NADDR 13
2
3 struct inode
4 {
5     char        i_flag;  État de l'inode en mémoire
6     char        i_count; /* reference count */ nb de utilisation de l'inode
7     dev_t       i_dev;   /* device where inode resides */ numéro device
8     ino_t       i_number; /* i number, 1-to-1 with device address */ numéro inode
9     unsigned short i_mode; type + droits
10    short        i_nlink; /* directory entries */ Nombre de lien
11    short        i_uid;   /* owner */ propriétaire
12    short        i_gid;   /* group of owner */ groupe propriétaire
13    off_t        i_size;  /* size of file */ taille
14    struct {
15        daddr_t    i_addr[NADDR]; /* if normal file/directory */ blocs données
16        daddr_t    i_lastr;       /* last logical block read (for read-ahead) */
17    };
18 };
19
```

dinode disque

inode mémoire

Table inode mémoire

i_flag = ILOCK i_count = 1 i_dev = 0 i_number = 3	0	
...		
i_flag = i_count = 2 i_dev = 0 i_number = 5	1	
...		
i_flag = i_count = 0 i_dev = i_number	2	libre
...		
i_flag = i_count = 1 i_dev = 0 i_number = 8	3	
...		
i_flag = i_count = 0 i_dev = i_number	4	libre
...		
...	...	

inode [NINODE]

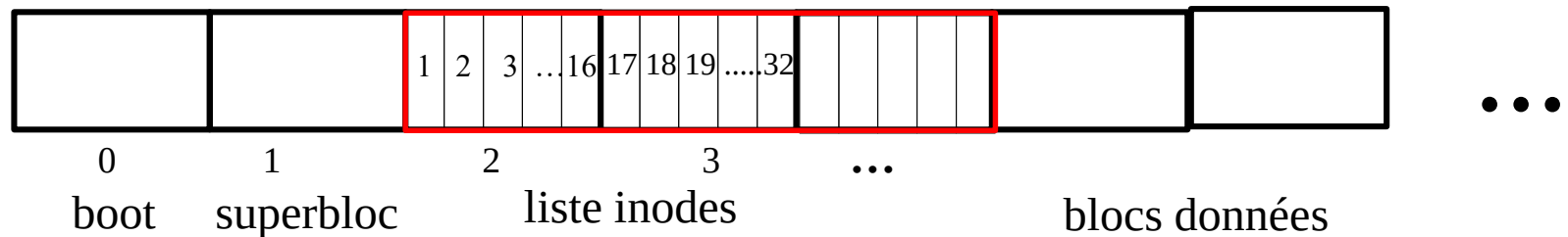
i_count :

- 0 : entrée libre
- $\neq 0$: nombre de référence à l'inode (open)

Table inodes disque

disque

16 dinodes par bloc



dinode

```
unsigned short i_mode;  
short         i_nlink;  
short         i_uid;  
short         i_gid;  
off_t         i_size;  
struct {  
    daddr_t    i_addr[NADDR];  
    daddr_t    i_lastr;  
};
```

$i_mode = 0 \Rightarrow$ inode libre

Structure inode/dinode (TD)

FICHER INODE.H

```
1 #define NADDR 13
2
3 struct inode
4 {
5     char        i_flag;
6     char        i_count; /* reference count */
7     dev_t       i_dev;   /* device where inode resides */
8     ino_t       i_number; /* i number, 1-to-1 with device address */
9     unsigned short i_mode;
10    short        i_nlink; /* directory entries */
11    short        i_uid;   /* owner */
12    short        i_gid;   /* group of owner */
13    off_t        i_size;  /* size of file */
14    struct {
15        daddr_t   i_addr[NADDR]; /* if normal file/directory */
16        daddr_t   i_lastr;        /* last logical block read (for read-ahead) */
17    };
18 };
19
```

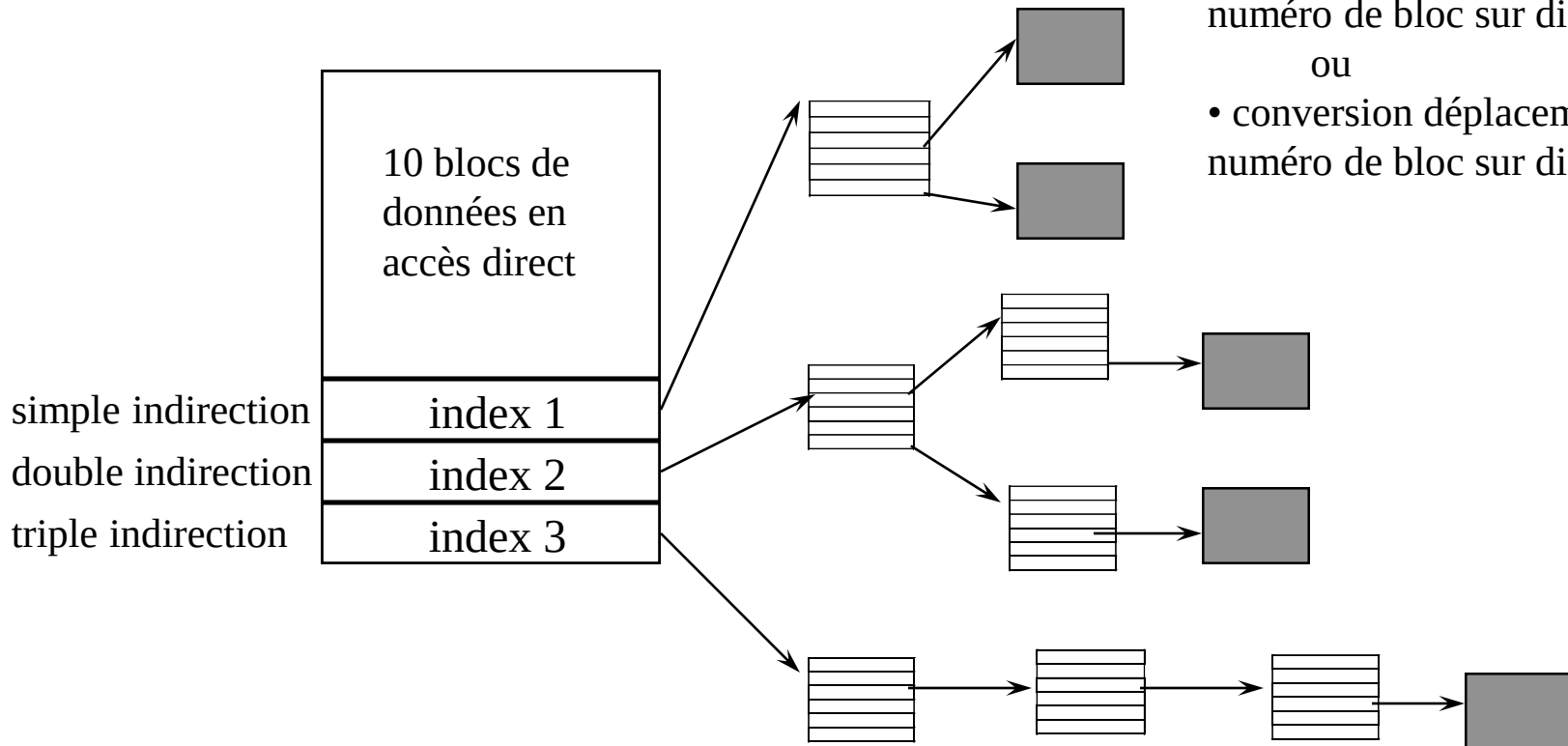
blocs données

dinode disque

inode mémoire

Où trouver les blocs ?

- Liste de blocs dans l'inode (**i_addr**)



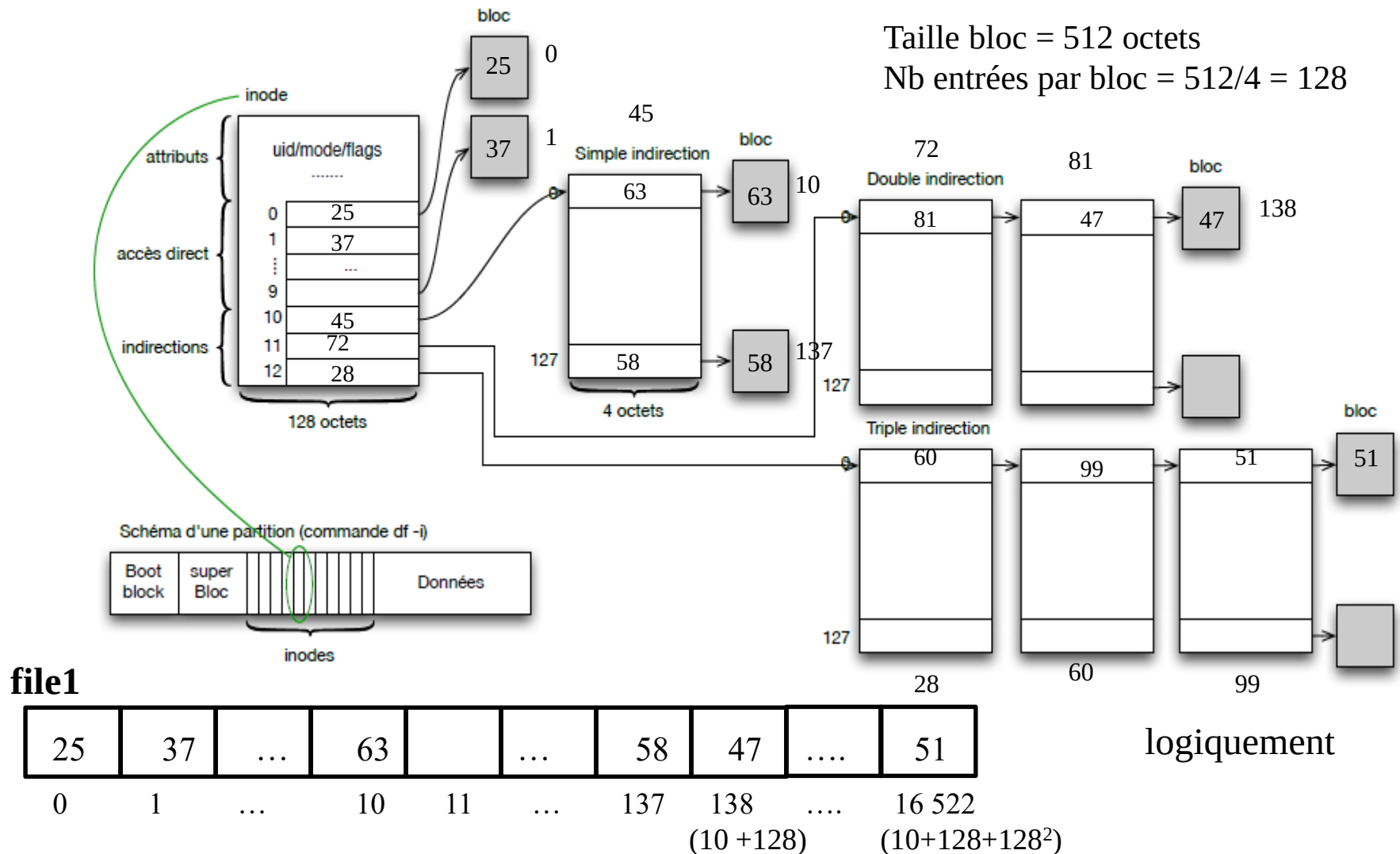
bmap

- conversion bloc logique -> numéro de bloc sur disque
- ou
- conversion déplacement -> numéro de bloc sur disque

- Vision de l'utilisateur :



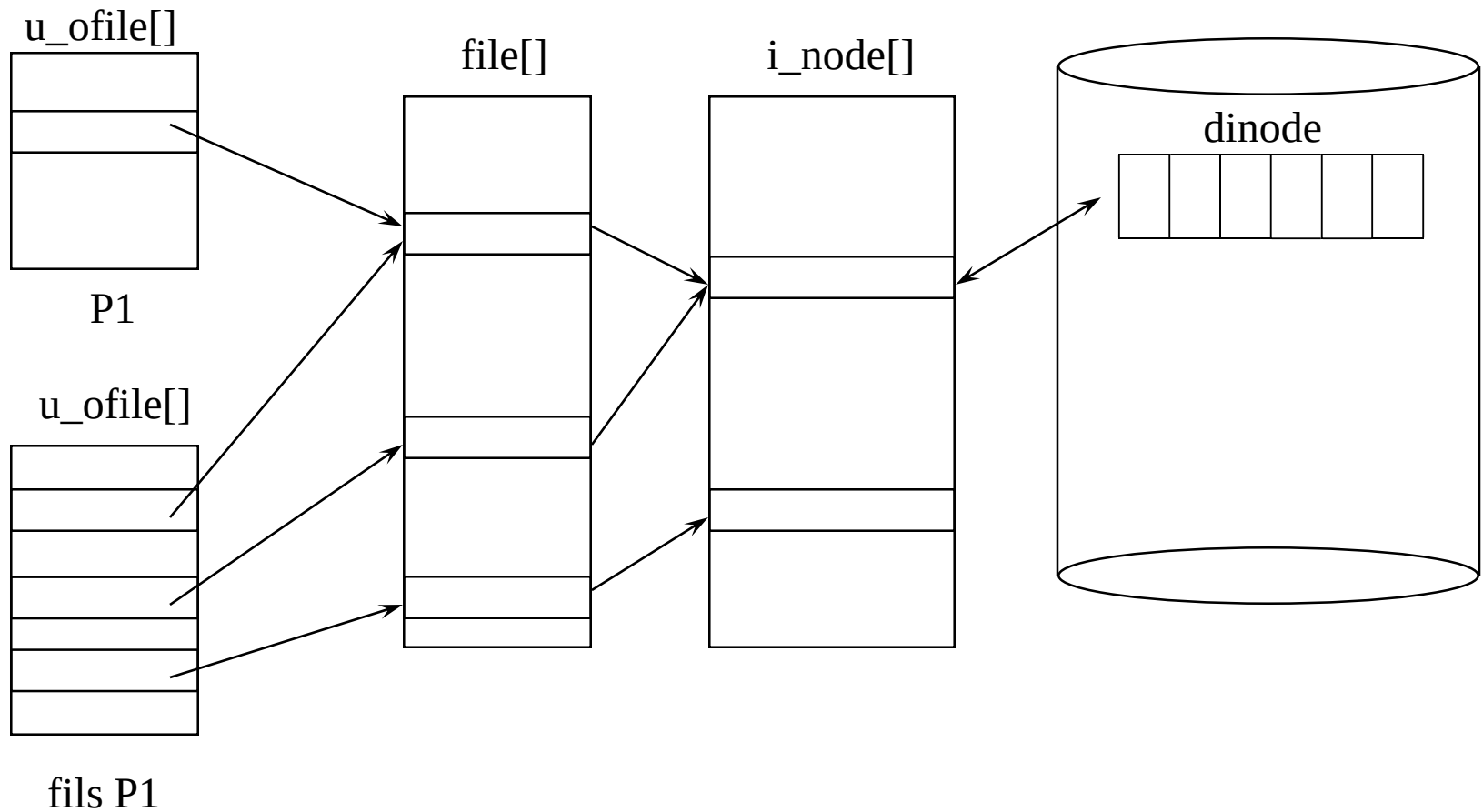
Bmap : bloc logique -> bloc sur disque



Les autres structures

- `file[]` : table globale des fichiers ouverts
 - `f_flag` : mode d'ouverture (Lecture, Ecriture, Lecture/Ecriture)
 - `f_offset` : déplacement dans le fichier
 - `f_inode` : numéro d'inode
 - `f_count` : nombre de références
- `u_ofile[]` : Table de descripteur locale des fichiers ouverts par un processus
 - pointeur vers entrée de `file [ ]`
 - `struct user` : processus élu

Résumé des structures



Fonction de manipulation des inodes

- `iget` : allouer/initialiser une nouvelle inode en mémoire (table inode [])
- `iput` : libérer l'accès à une inode en mémoire (table inode [])
- `namei` : retrouve une inode à partir d'un nom de fichier (appeler par `open`)
- `ialloc` : allouer une nouvelle inode disque à un fichier (`creat`)
- `ifree` : détruire une inode sur disque (`unlink`)

Principe de iget

iget (dev, ino)

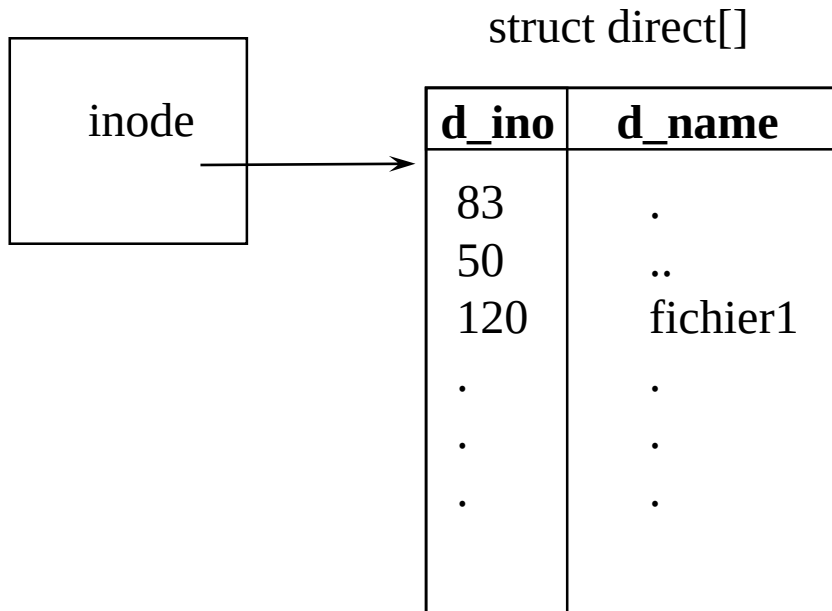
- Chercher l'inode (dev, ino) dans la table inode []
- Si inode présente et verrouillée
 - Attendre libération; recommencer recherche
- Si inode non présente dans inode []
 - Chercher une entrée libre dans la table inode []
 - Remplir l'entrée à partir de l'inode sur disque
- Incrémenter le nombre de référence
- Verrouillé l'entrée
- rendre pointeur vers l'entrée

i_flag = ILOCK i_count = 1 i_dev = 0 i_number = 3 ...	0
i_flag = i_count = 2 i_dev = 0 i_number = 5 ...	1
i_flag = i_count = 0 i_dev = i_number ...	2
i_flag = i_count = 1 i_dev = 0 i_number = 8 ...	3
i_flag = i_count = 0 i_dev = i_number ...	...

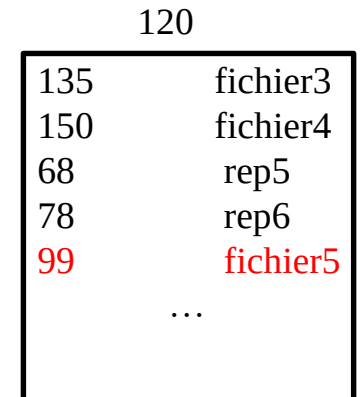
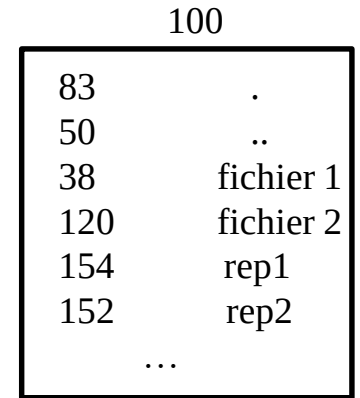
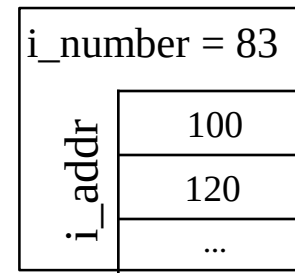
inode [NINODE]

Les répertoires

- Répertoire = un **fichier** de type répertoire
=> bloc de données de l'inode



d_name (14 caractères) SVR4
(255 caractères) BSD



Algorithme de namei

Entrées: nom du chemin

Sortie: inode

Si (premier caractère du chemin == '/')

 dp = inode de la racine (rootdir) (iget pour verrouiller)

sinon

 dp = inode du répertoire courant (u.u_cdir) (iget pour verrouiller)

Tant qu'il reste des constituants dans le chemin {

 lire le nom suivant dans le chemin

 vérifier les droits et que dp désigne un répertoire

 si dp désigne la racine et nom = ".."

 continuer

 lire le contenu du répertoire (bmap pour trouver le bloc puis bread et brelse)

 si nom suivant appartient au répertoire {

 libérer dp (iput);

 dp = inode correspond au nom (iget)

 }

 sinon // Pas d'inode

}

retourner dp

Exemple 1

namei("../rep1/f1")

①
u.u_cdir

inode []

i_number = 14	
i_addr	100

③
iget

i_number = 20	
i_addr	300

⑤
iget

i_number = 30	
i_addr	500

bloc 100

"."	14
".."	20

② bread

bloc 300

"."	20
"rep1"	30

④ bread

bloc 500

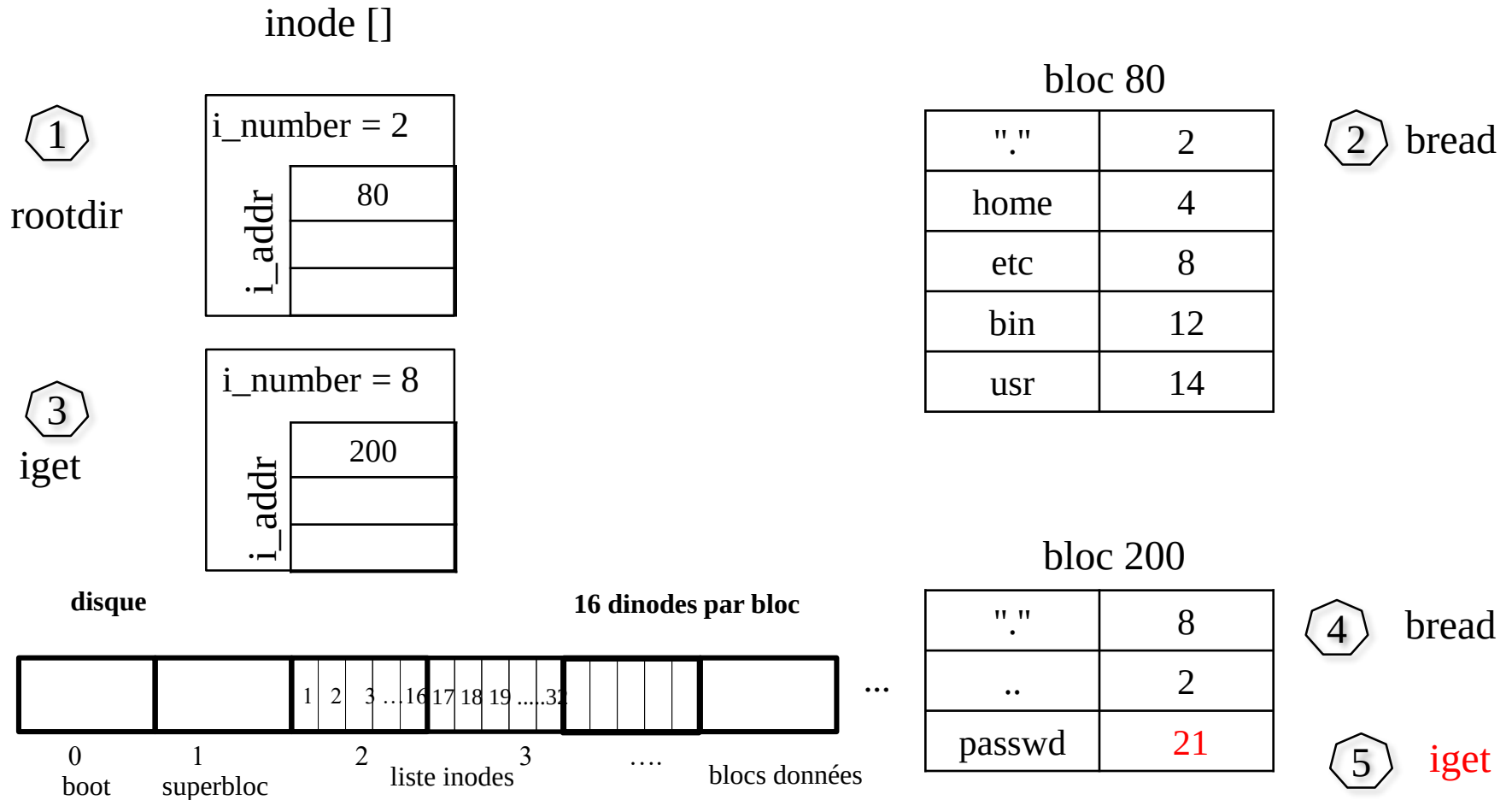
"."	30
"f1"	320

⑥ bread

⑦ iget

Exemple 2

namei("/etc/passwd")



Les liens

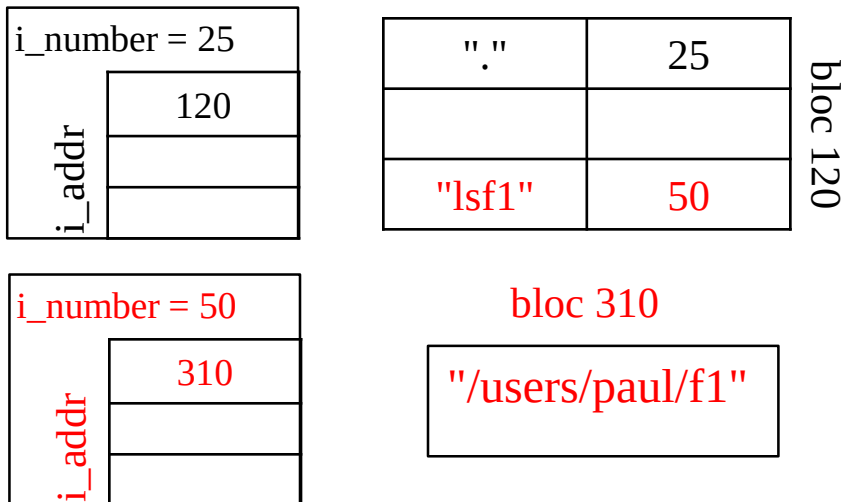
- Fichiers spéciaux
 - Liens symboliques : contiennent le nom du fichier source
 - Liens physiques : désignent la même inode que le fichier source
- Accès liens symboliques
 - Droits : 1) droits sur le lien
 - 2) droits sur le fichier source
- Accès liens physiques
 - Droits : droits sur le fichier

Liens symbolique et physique: exemple

1 fichier source `/users/paul/f1` d'inode **35** ; répertoire `/users/pierre` d'inode 25
répertoire `/users/paul` d'inode 15

Symbolique

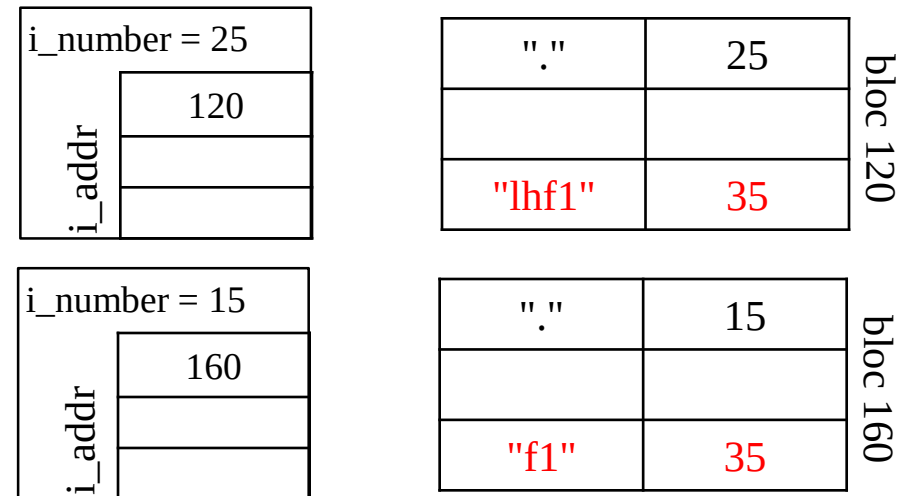
`ln -s /users/paul/f1 /users/pierre/lfs1`



allocation nouveau inode : 50

Physique

`ln /users/paul/f1 /users/pierre/lhf1`



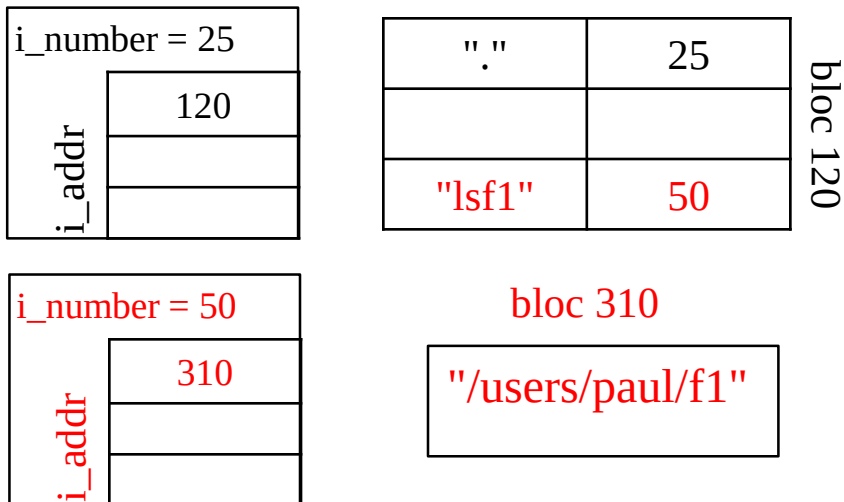
incrémenter nombre de lien inode 35

Liens symbolique et physique: exemple

1 fichier source `/users/paul/f1` d'inode **35** ; répertoire `/users/pierre` d'inode 25
répertoire `/users/paul` d'inode 15

Symbolique

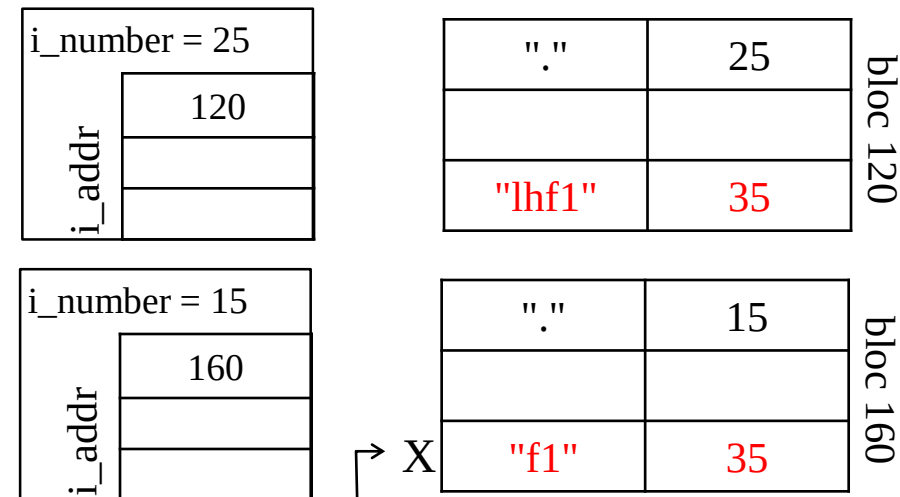
`ln -s /users/paul/f1 /users/pierre/lfs1`



`cat /users/pierre/lfs1`
No such file or directory

Physique

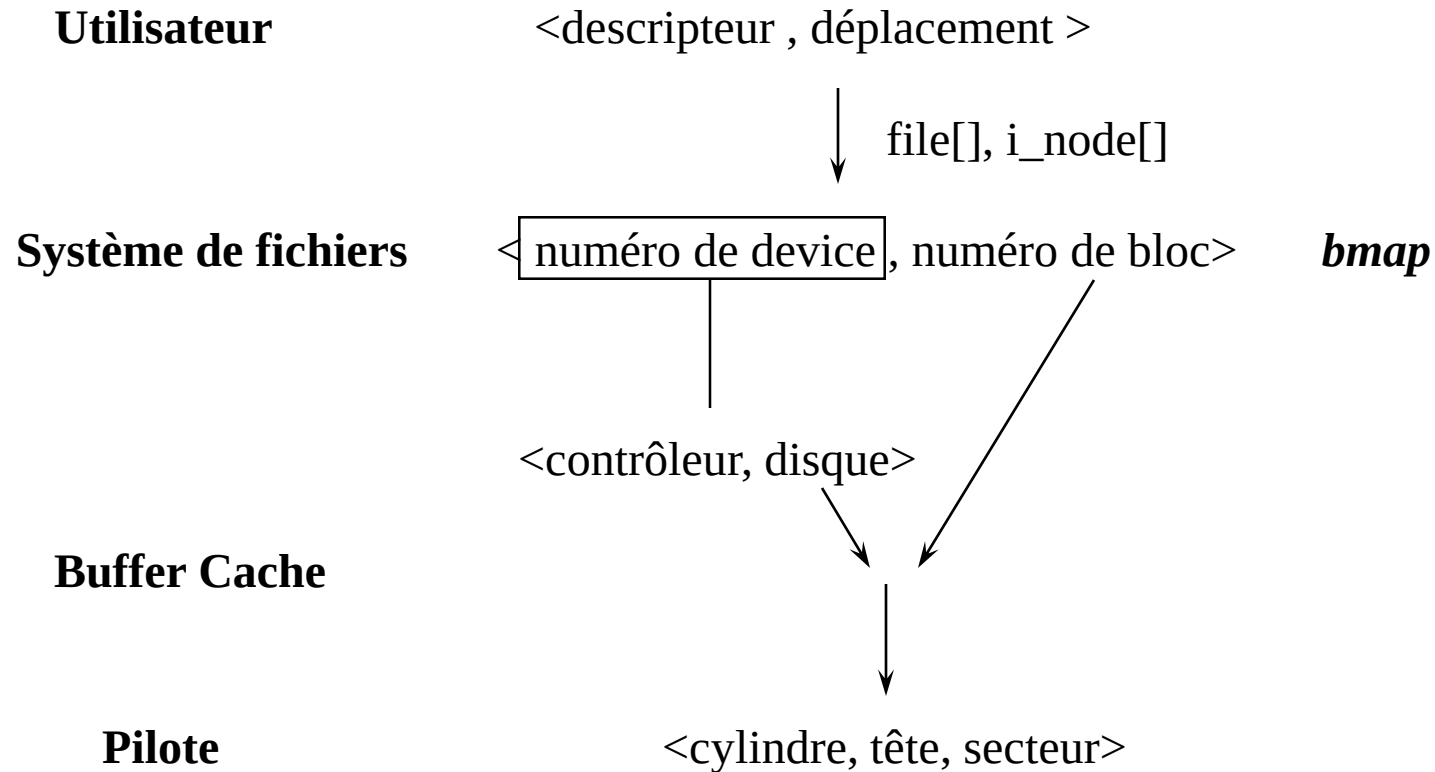
`ln /users/paul/f1 /users/pierre/lhf1`



`rm /users/paul/f1` → X
`cat /users/pierre/lhf1`
...

Implémentation des appels système

- Systèmes d'adressage :

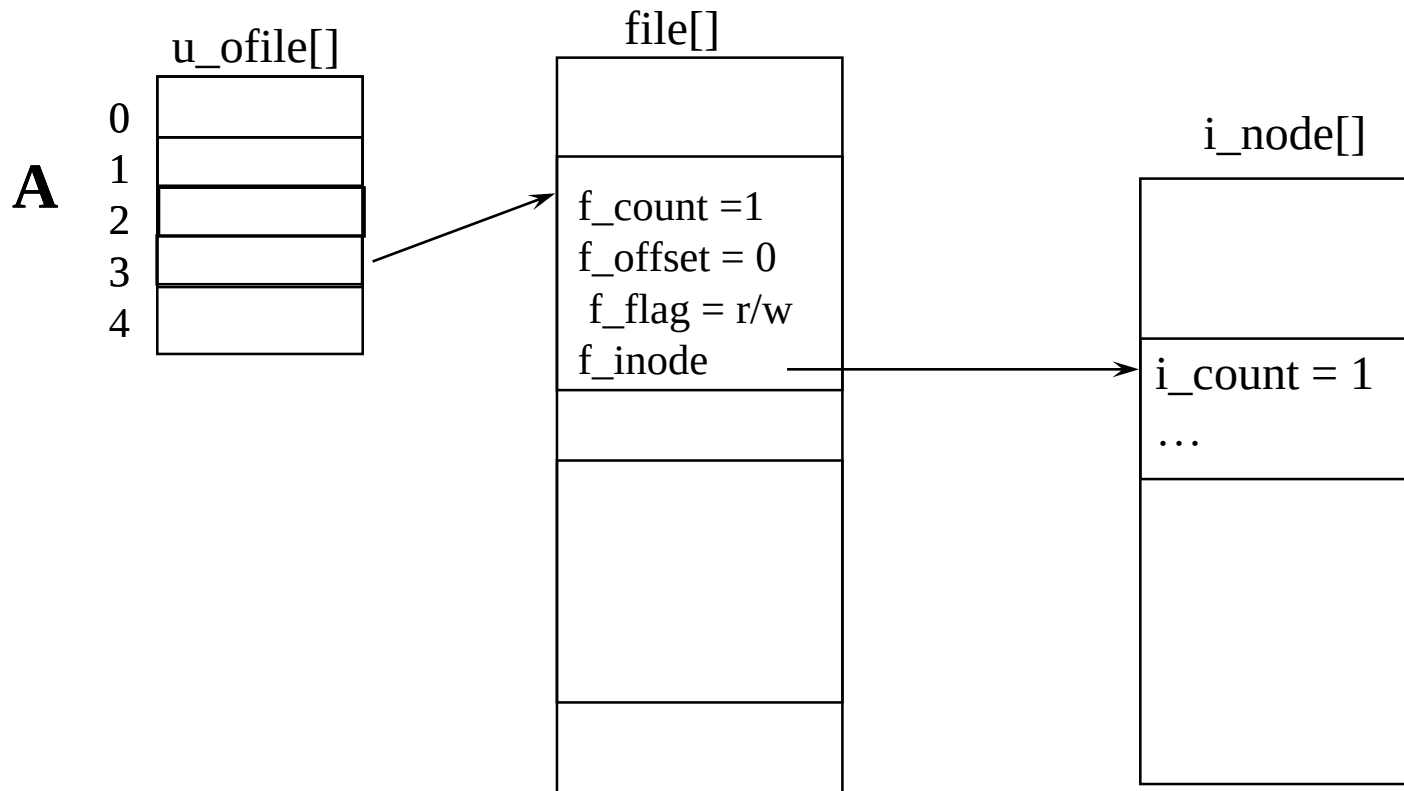


Algorithme de open

- Retrouver l'inode à partir du nom de fichier (namei)
- Si (fichier inexistant ou accès non permis) retourner erreur
- allouer et initialiser un élément dans la table file[]
- allouer et initialiser une entrée dans u_ofile du processus
- Si (mode indique une troncature) libérer les blocs (free)
- déverrouiller inode
- retourner le descripteur

Exemple 1

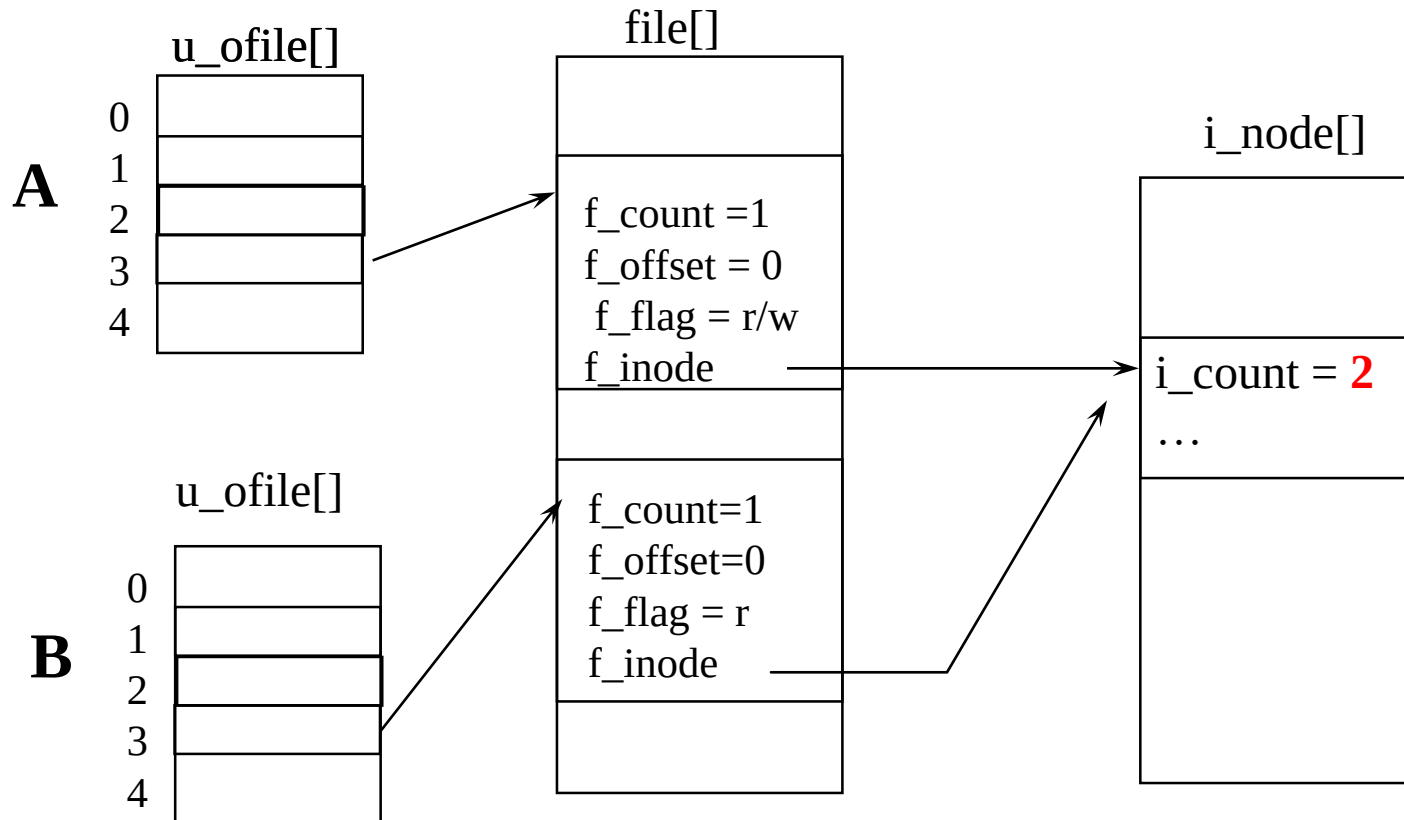
Processus A : `fd = open ("/home/sens/monfichier", O_RDWR|O_CREAT, 0666);`



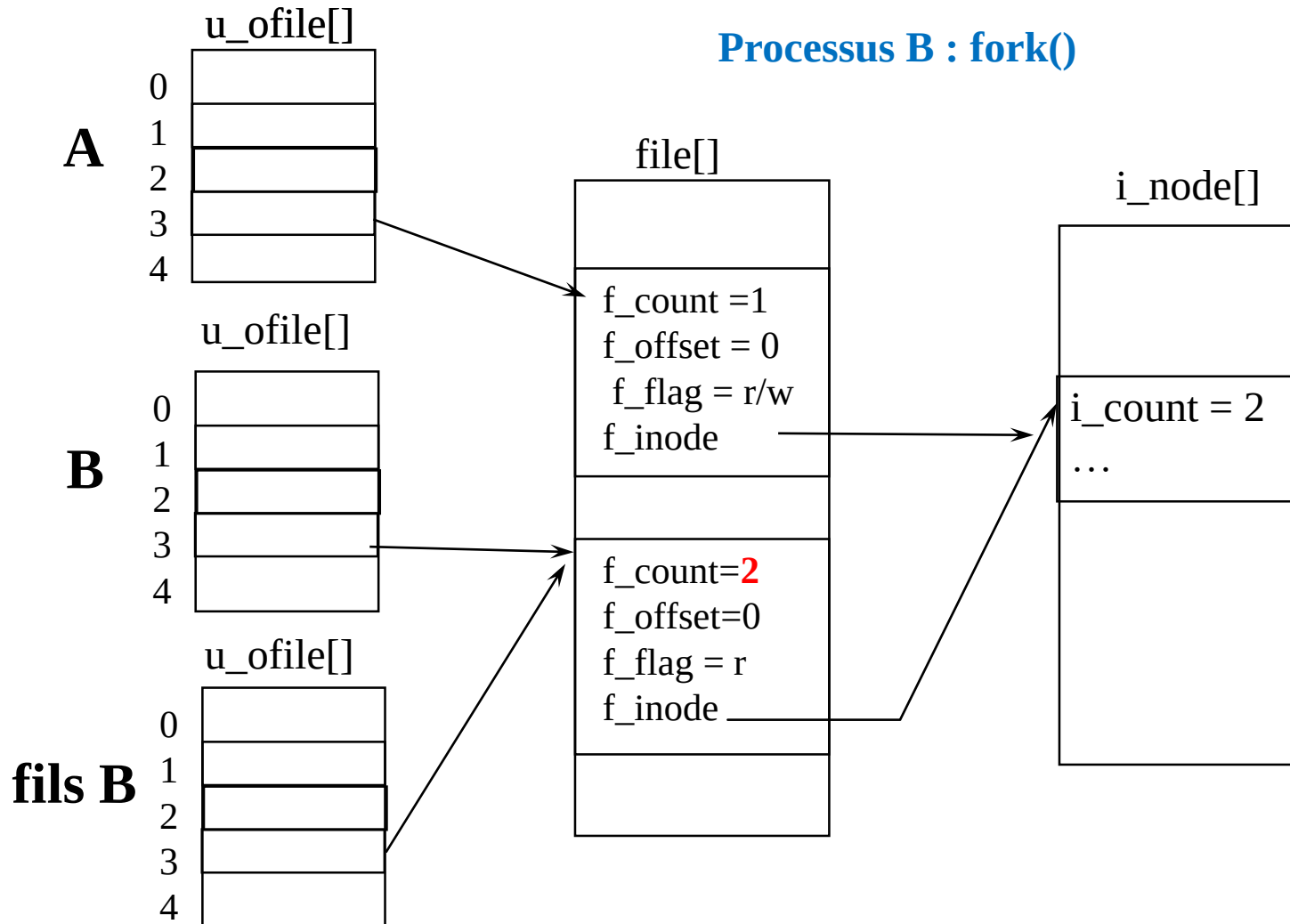
Exemple 1

Processus A : `fd = open ("/home/sens/monfichier", O_RDWR|O_CREAT, 0666);`

Processus B : `fd = open ("/home/sens/monfichier", O_RDONLY);`



Exemple 1

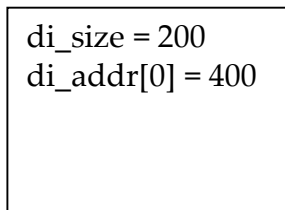


Exemple 2

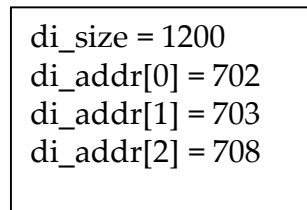
Processus **P**

u.u_cdir, désigne l'inode numéro **10** (bloc 2)

inode 10 la seule chargée en mémoire

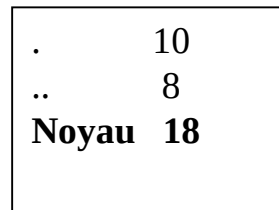


i_number = 10

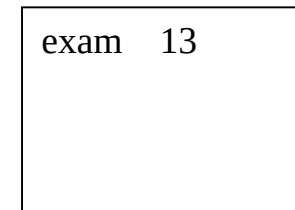


i_number = 18

inodes



bloc 400



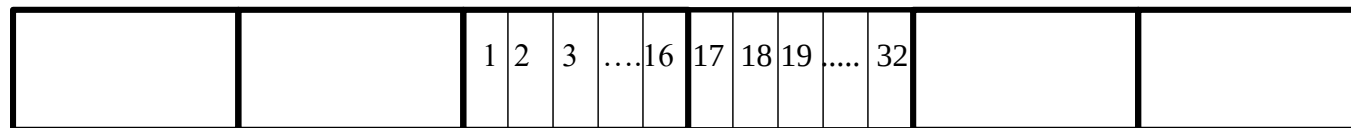
bloc 708

Blocs de données

Fichier : **Noyau/exam**

disque

16 inodes par bloc



0

1

2

3

....

boot

superbloc

liste dinode

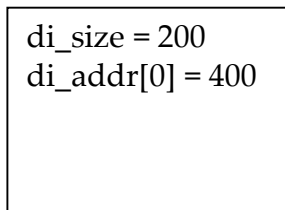
blocs données

...

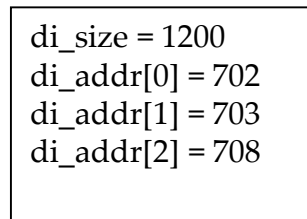
Exemple 2

```
fd = open("Noyau/exam," O_RDWR) ;
```

open : *namei* pour déterminer l'inode du fichier puis *iget* pour charger l'inode dans la table d'inode en mémoire.

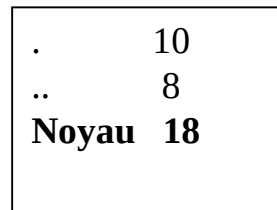


i_number = 10

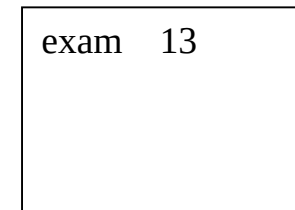


i_number = 18

inodes



bloc 400

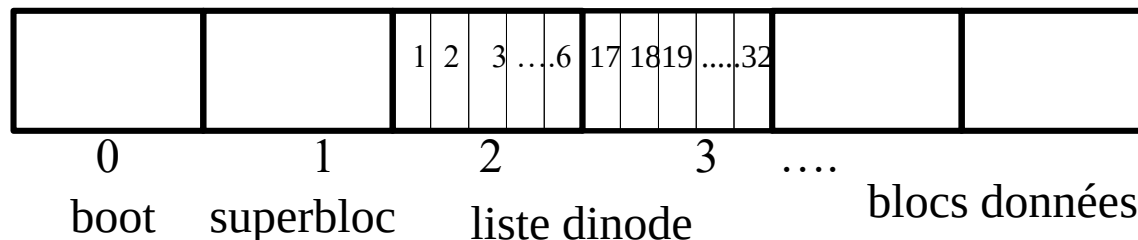


bloc 708

Blocs de données

disque

16 inodes par bloc

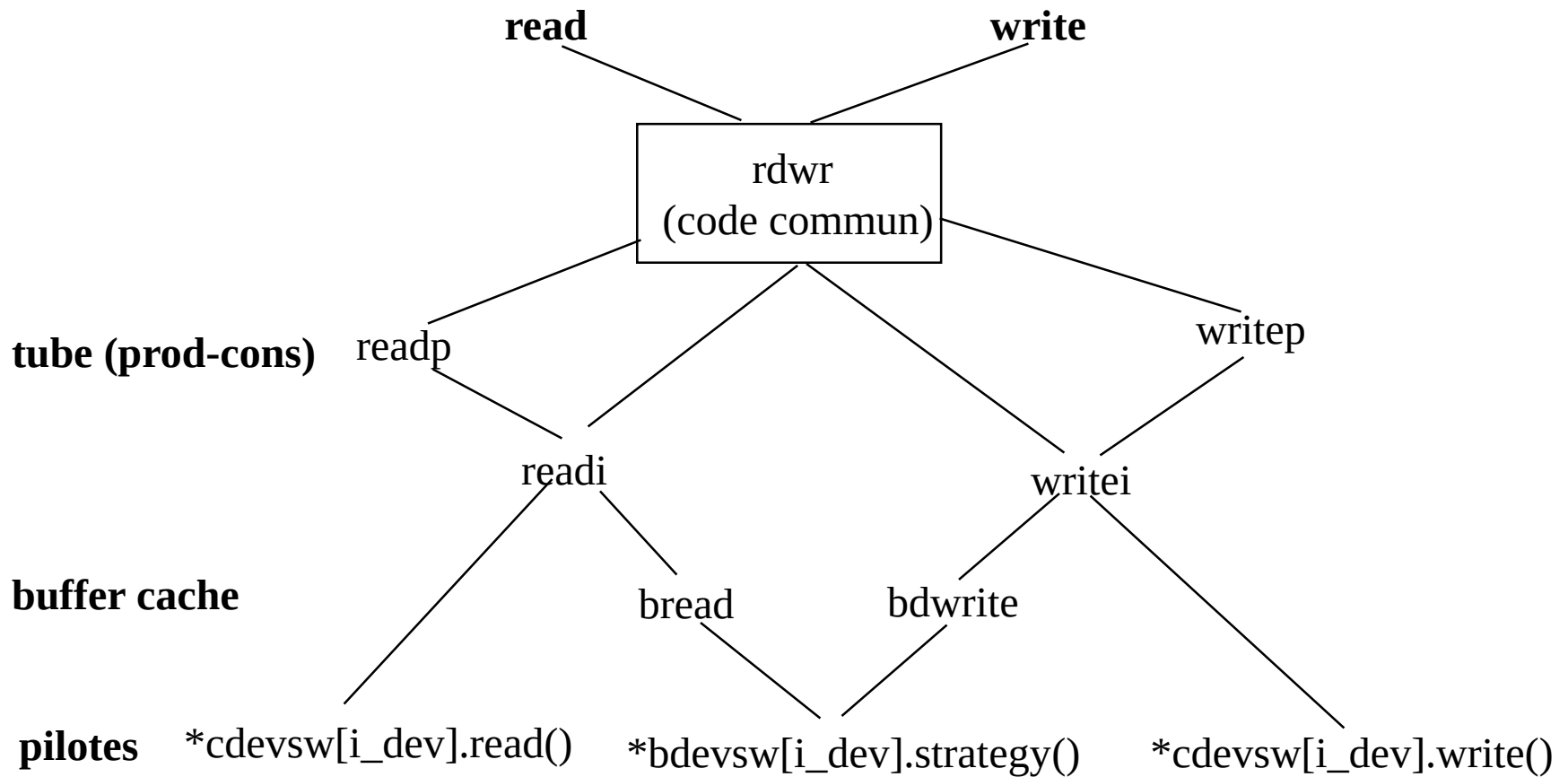


...

5 E/S Blocs:

400
3
702
703
708

Appels read et write



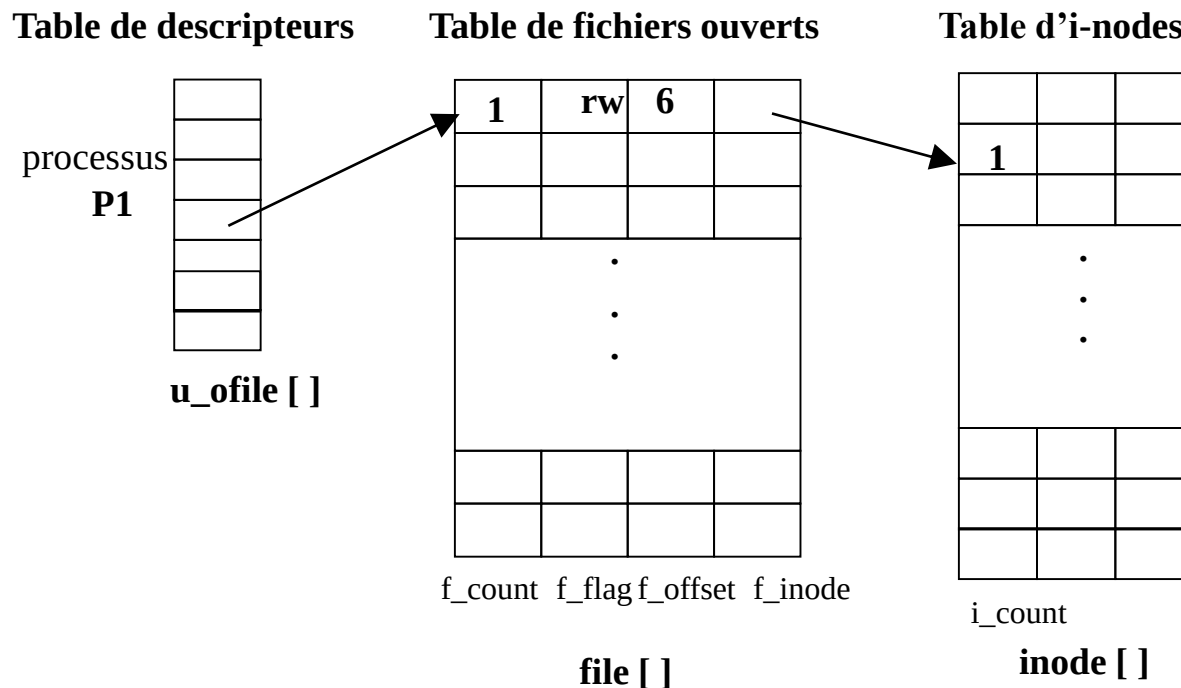
Algorithme de read(fd,buf,size)

- Accéder à l'entrée de file[] à partir de u_ofile[fd]
- Vérifier le mode d'ouverture (champs f_flag)
- Copier dans la zone u les informations pour le transfert
- Verrouiller l'inode (f_inode)
- Tant que (nombre octets lus < nombre à lire)
 - Conversion déplacement numéro bloc (bmap)
 - Calculer le déplacement dans le bloc
 - Si (nb octets restants == 0) break; // Fin de fichier
 - Lecture du bloc dans le cache (bread)
 - Transférer tampon dans zone u
 - libérer le tampon verrouiller par bread (brelse)
- Déverrouiller l'inode;
- Mettre à jour file[]
- Retourner nombre octets lus

Exemple 1

P1 :

```
int main (int argc, char* argv []) {  
    int fd1;  
    fd1 = open ("file" O_RDWR|O_CREAT, 0666) ;  
    write (fd1,"abcdef", strlen ("abcdef")) ;  
}
```



Exemple 1

P1 :

```
int main (int argc, char* argv []) {
    int fd1,
    fd1 = open ("file" O_RDWR| O_CREATE,0666);
    write (fd1,"abcdef", strlen ("abcdef"));
}
```

P2:

```
int main (int argc, char* argv []) {
    int fd1,
    fd1 = open ("file" O_RDWR);
    write (fd1,"123", strlen ("123"))
}
```

Table de descripteurs

processus
P1

Table de descripteurs

processus
P2

Table de fichiers ouverts

1	rw	6	
1	rw	3	
.			
.			
.			

f_count f_flag f_offset f_inode

file []

Table d'i-nodes

2		
.		
.		
.		

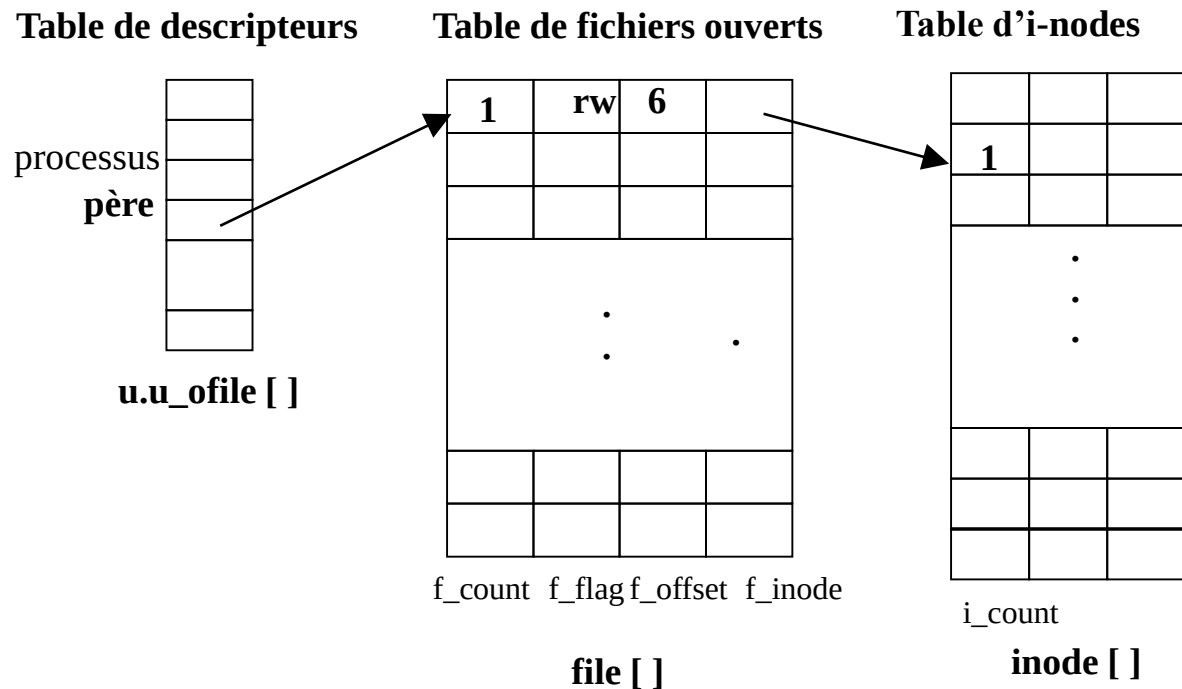
i_count

inode []

Example 2

```
int main (int argc, char* argv []) {
    int fd1,
    fd1 = open ("file" O_RDWR| O_CREAT,0666) ;
    write (fd1,"abcdef", strlen ("abcdef")) ;
    if (fork ( ) == 0)
        write (fd1,"123", strlen ("123"));
}
```

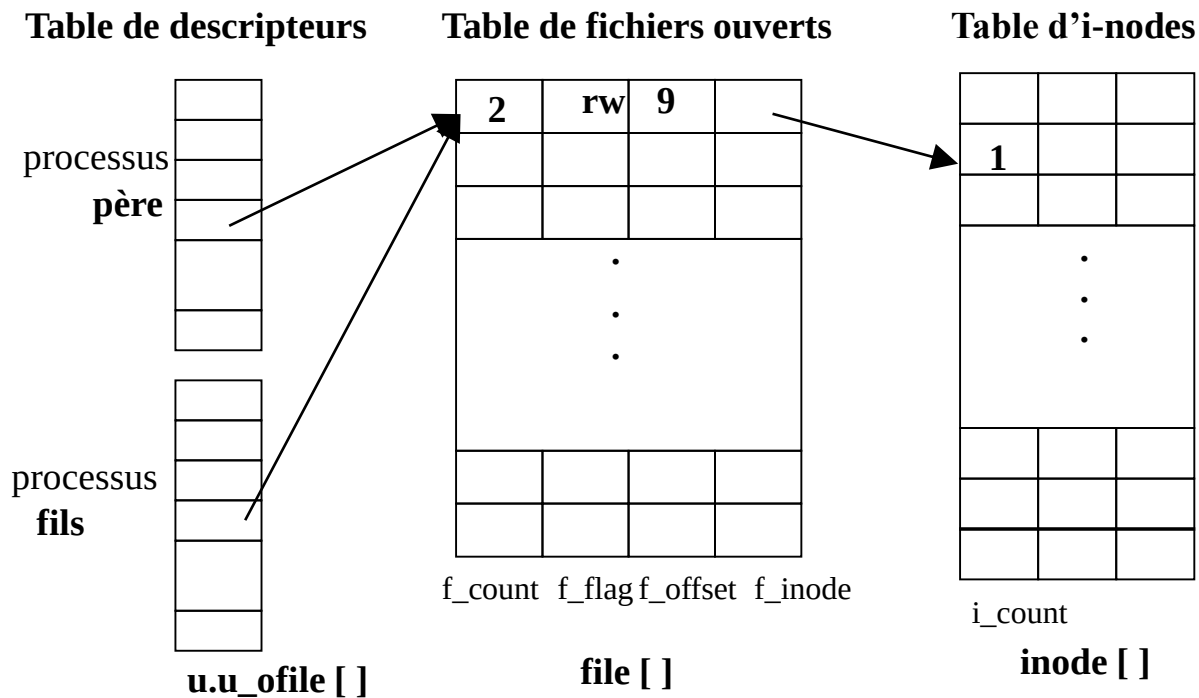
```
file :
abcdef
```



Exemple 2

```
int main (int argc, char* argv []) {
    int fd1,
    fd1 = open ("file" O_RDWR| CREATE) ;
    write (fd1,"abcdef", strlen ("abcdef"));
    if (fork ( ) == 0)
        write (fd1,"123", strlen ("123"));
}
```

file :
abcdef123

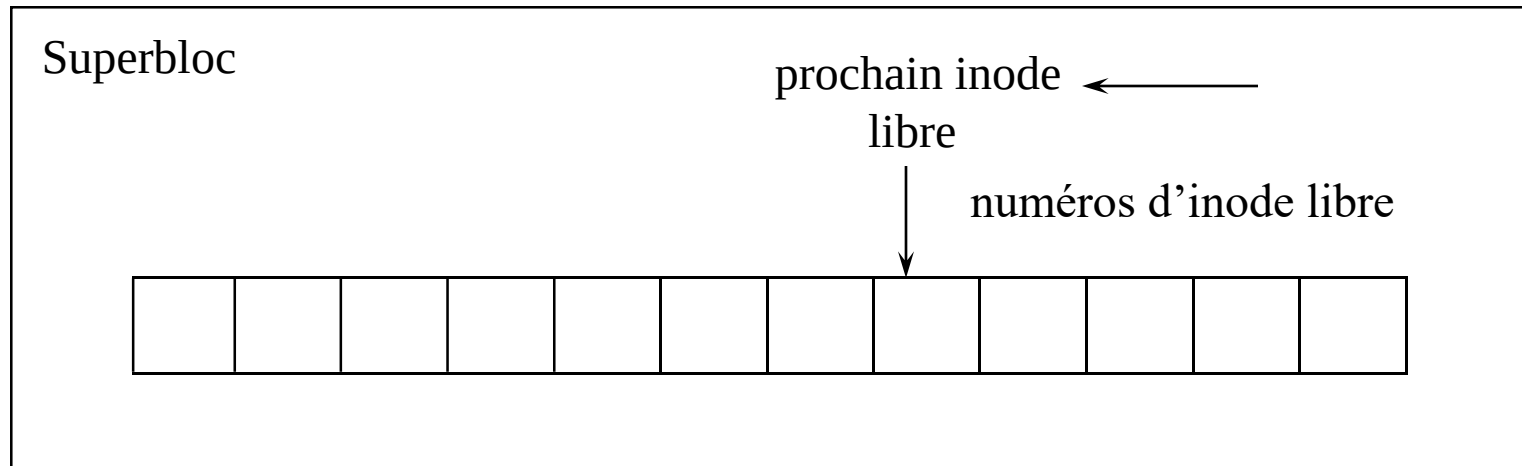


Gestion des inodes/blocs libres

- Le superbloc : struct fs (fs.h)
Bloc d'administration de la partition qui contient :
 - Taille en blocs du système de fichier
 - Taille en blocs de la table des inodes
 - Nombre de blocs libres et d'inodes libres
 - Liste des blocs libres
 - Liste des inodes libres sur disque

Gestion des inodes libres sur disque

- Le superbloc contient une liste partielle des inodes libres

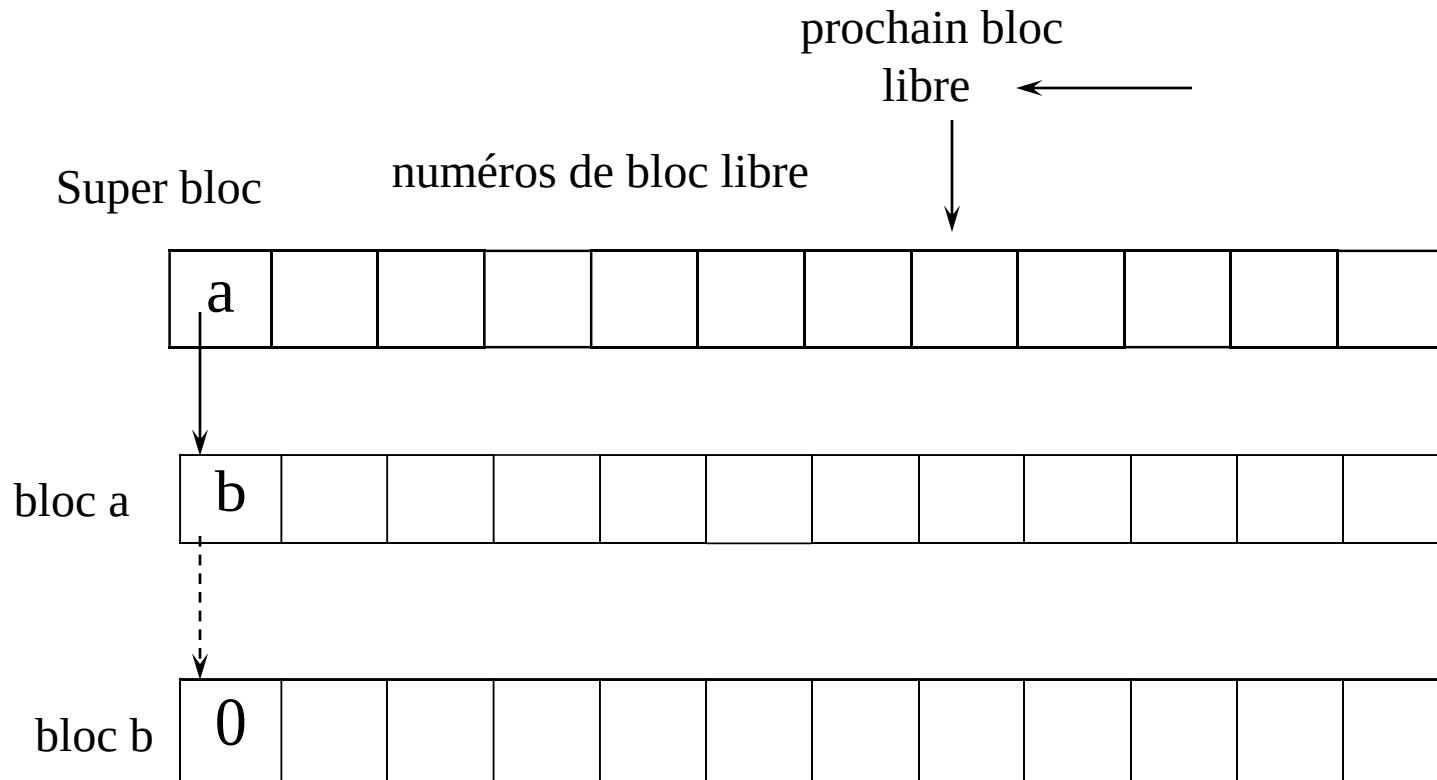


- Si liste vide, réinitialiser la liste en «scannant» la table des inodes sur disque

Principe de ialloc

- Vérifier si aucun autre processus n'a verrouillé le superbloc (sinon sleep)
- Verrouiller le superbloc
- Si liste des inodes libres sur disque non vide
 - Prendre l'inode libre suivante dans superbloc
 - attribuer une inode en mémoire (iget)
 - mise à jour sur disque (inode marquée prise)
- Si liste vide
 - Verrouiller le superbloc
 - parcourir la liste des inodes sur disque pour remplir le superbloc
- Tester à nouveau si l'inode est vraiment libre sinon la libérer et recommencer (conflit d'accès à un même inode !)

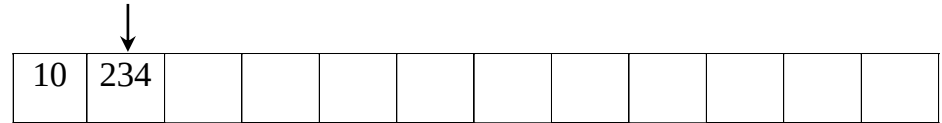
Allocation des blocs libres



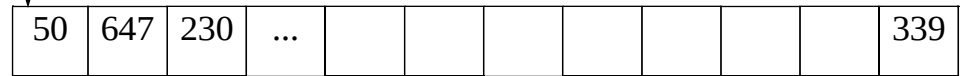
Allocation/libération de bloc : exemple

Configuration initiale

Superbloc

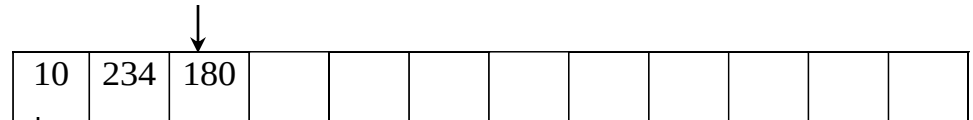


bloc 10



Libération du bloc 180

Superbloc

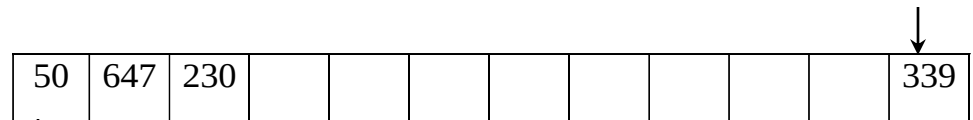


bloc 10

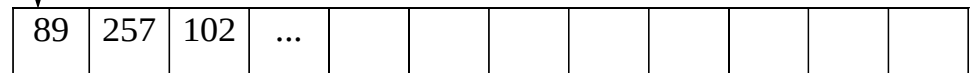


Allocation de 3 blocs (180, 234, 10)

Superbloc

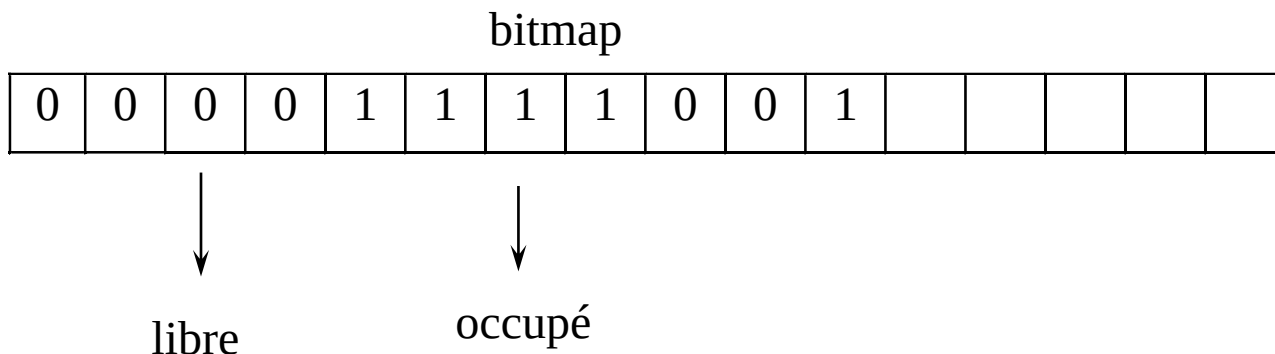


bloc 50



Allocation dans les nouveaux systèmes de fichiers

- Pb de la stratégie "classique" : pas de prise en compte de la contiguïté des blocs libres
- => vecteur binaire



- Exemple Ext2fs

Les optimisations : fast file system (ffs)

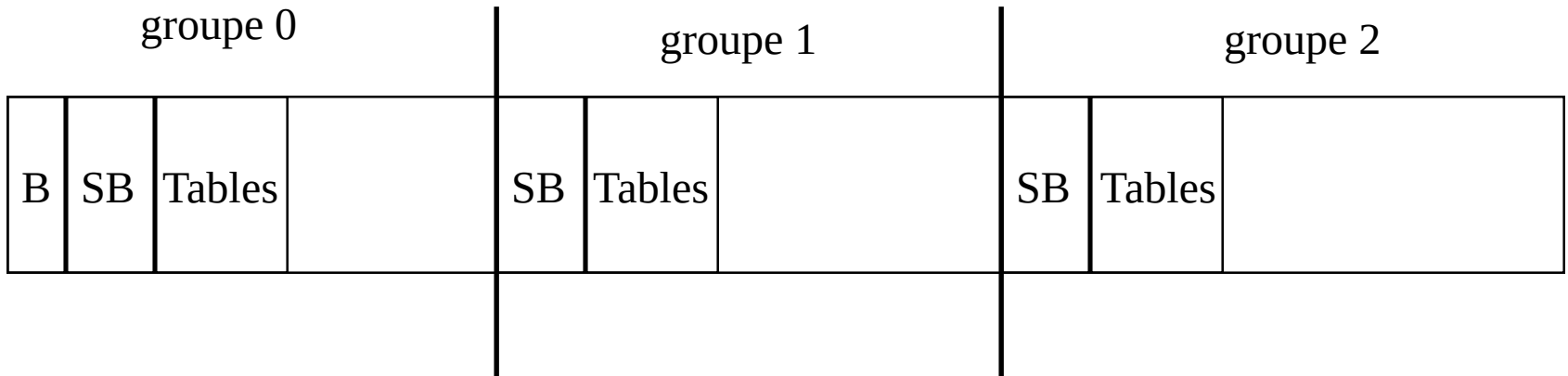
- Intégrer dans tous les unix (connu comme ufs)
- De nombreuses améliorations

=> Augmenter la fiabilité

=> Augmenter les performances

Organisation en groupe

- Disque divisé en groupe de cylindre



- Réplication du superbloc => augmenter la fiabilité
- Dissémination des tables => réduction des temps d'accès

Blocs et fragments

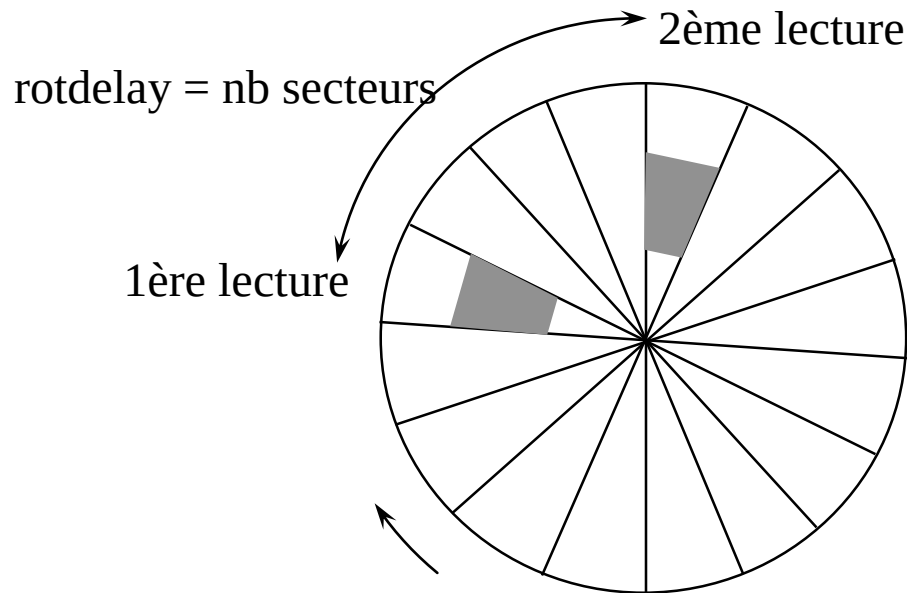
- Problème sur la taille des blocs
 - Taille de blocs importante => plus de données transférées en une E/S
plus d'espace perdu (1/2 bloc en moyenne)
- Idée : partager les blocs entre plusieurs fichiers
- => Blocs divisés en fragment
 - 2,4,8 fragments par bloc
 - Taille "classique" : blocs 8Ko, fragment 512 octets
- Unité d'allocation = fragment
 - => perte réduite
 - => plus de structures de données

Optimisations

- Optimisations :
 - 1) Regrouper toutes les inodes d'un même répertoire dans un même groupe
 - 2) Inode d'un nouveau répertoire sur un autre groupe
=> distribution des inodes
 - 3) Essayer de placer les blocs de données d'un fichier dans un même groupe que l'inode
- => limiter les déplacements de tête

Politique d'allocation de bloc

- Constatations : la plupart des lectures sont séquentielles
- => placement des blocs d'un même fichier
 - En fonction de la vitesse de rotation pour optimiser lecture séquentielle
 - Objectif : faire en sorte que lors de la lecture suivant le bloc soit sous la tête

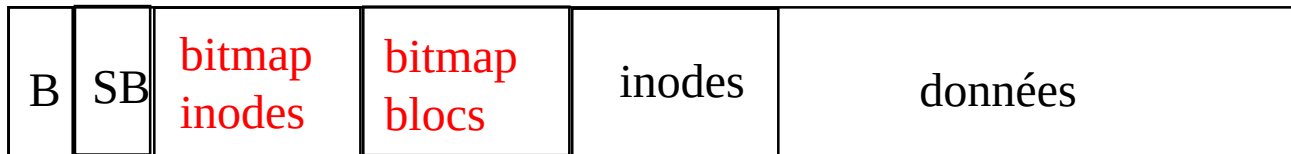


Performances

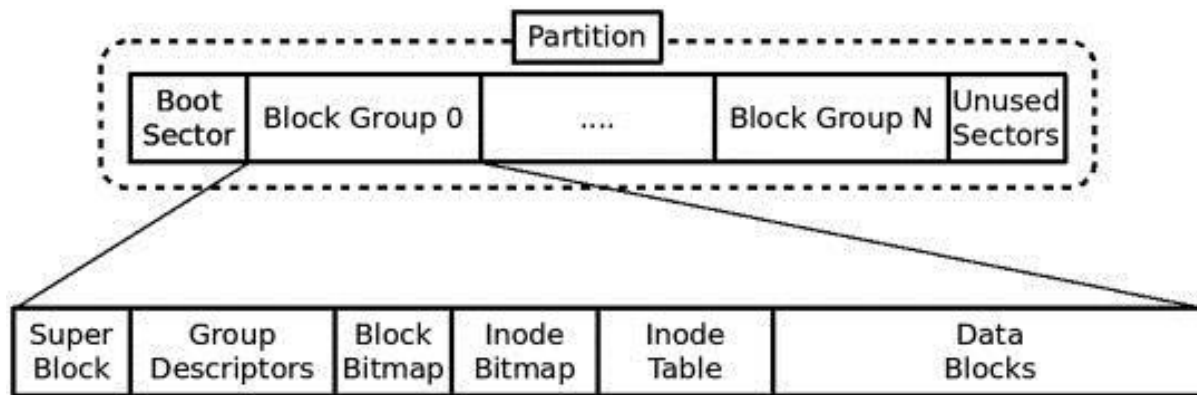
- Stratégie d'allocation efficace si disque pas trop plein (< 90%)
- Sur VAX/750
- Accès lecture débit = 29 Ko/s s5fs
débit = 221 Ko/s ffs
- Accès écriture débit = 48Ko/s s5fs
débit = 142 Ko/s ffs

D'autres organisations

Ext2fs



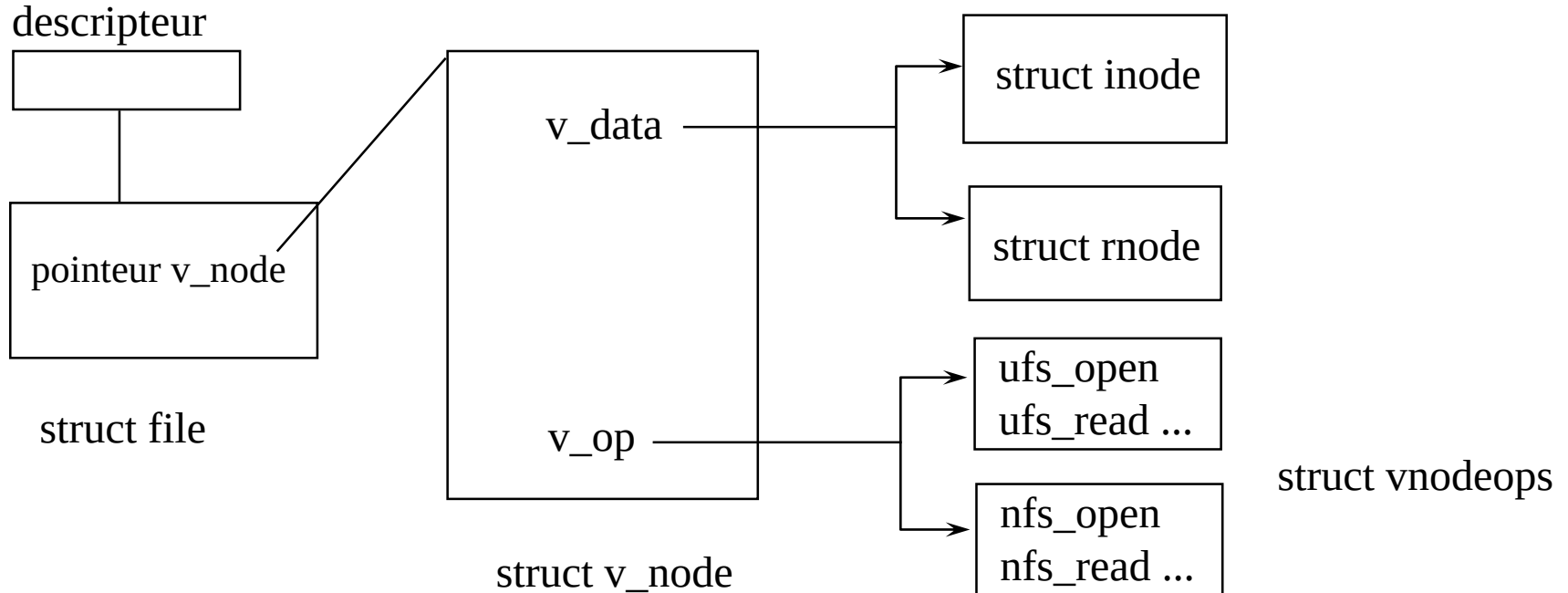
Ext4fs



ajout de notion d' **extend** : blocs consécutifs pour gros fichiers
=> limiter les indirections

Systemes génériques : VFS

- Objectifs : gérer différents systèmes de fichiers locaux et distants => Virtual File System
- Ajout d'une couche supplémentaire responsable de l'aiguillage : couche vnode (virtual node)



Architecture VFS

